MUSTER: Top-Ranked, Self-Configuring Agent Memory That Knows a Log from a Novel and Retrieves Each by Its Structure

Woosang Lim
SAINT Machines and KAIST
saintmachines.github.io/MUSTER
github.com/saintmachines/MUSTER



Figure 1: **Self-configuring MUSTER scores highest and builds fastest.** Values from Tables 7 and 16.

ABSTRACT

An AI agent's memory stores differ in the structure they already record. Logs number their lines, example banks separate their records, and prose does neither. RAG keeps every word and can score well after heavy tuning to a dataset, but the tuning does not generalize. Model-built memory scales to long histories but is costly to build and can drop evidence. Agents need an autonomous strategy that is cheap to build and leaves little to tune. MUSTER detects each store's structure and builds memory from it. Two byte statistics, the shares of numbered lines and of text in records short enough to keep whole, set how a store is cut, grouped and ordered, with no model, labels, queries or candidate runs. MUSTER matches retrieval to that structure, which sets how dense, lexical and parent-graph channels split the slots. It assembles the context in stored order, keeping the newest version of each fact where an order is recorded. The rule for cutting and grouping, set on MemoryAgentBench, applies unchanged to LoCoMo and BEAM. On MemoryAgentBench~~the released implementation with the paper settings~~, the SDK's collection Auto configuration (version 0.1.3) has the highest Overall among the retrieval and memory systems at the benchmark's reader (~~60.5~~ 60.6 against 43.6). ~~Its per-store automatic configuration scores 58.5.~~ The original implementation is the strongest row on LoCoMo under both answer prompts (86.04 and 93.46) with no generative call before the answer, where the strongest other system makes 9,398. On LoCoMo-Refined it matches that system under the official evaluation (77.57 against 75.33) and leads it by 8.5 points with a reasoning reader. On BEAM, from 100K to 10M tokens, it matches the strongest system we compare at equal context and has the highest mean with an open-weight reader at 9.7K tokens. It builds a 10M-token memory in 77 seconds, where model-built memories take hours. Read beside a model-built memory, it averages 73.1 on BEAM at 19.5K tokens, above either store alone and every system at that budget. MUSTER keeps each store as written and re-derives its configuration in seconds as the store grows.

1 INTRODUCTION

An agent that answers from memory inherits a store it did not design. The store arrives as bytes (a transcript, a list of assertions, a novel, a bank of labelled examples), and something must decide how to cut it, how to group the pieces, and what to do when two pieces disagree. RAG keeps every word and can score well after heavy tuning to a dataset, but its recipe is hidden, and finding it takes labelled questions or candidate runs for every store. Model-built memory scales to long histories but

pays before the first question and can drop evidence [27]. An agent that keeps logs, example banks and documents beside its conversations needs an autonomous strategy that is cheap to build, leaves little to tune and first asks whether a store already records the structure a reader needs.

In MemoryAgentBench [1], fact-consolidation stores number their assertions, in-context-learning stores separate their examples with blank lines, and narrative stores do neither. MUSTER measures these properties (Section 3.1, Figure 2). serial detects numbered records that expose an update order, and recshare tells whether bounded records already make good retrieval units. The two statistics see no question, gold answer or dataset name and call no model~~, and over fourteen stores~~. In the original evaluation, they run in three seconds over fourteen stores and configure thirteen (LongMemEval keeps its sessions).

The memory systems we run are reported on conversation benchmarks alone, and their lead does not carry. The strongest other system we ran on LoCoMo scores 23.1 on MemoryAgentBench, below BM25's 41.5 (Table 1). The rule set on MemoryAgentBench derives the units and parents of LoCoMo, LoCoMo-Refined, BEAM and a model-built memory from their bytes, with no change to the rule. In the results of Section 5, MUSTER is the strongest memory system on each benchmark or matches it at equal context.

Detect before you construct. Unlike True Memory [30], which reads every store with one fixed representation, and Adaptive Chunking [2], which picks a chunker by running every candidate, MUSTER derives a store's unit, parent, order and retrieval policy from two byte statistics, with no model and no candidate run. It *detects* whether the store records the structure a reader needs, including an order, and sets the unit and hierarchy from it. It *matches* a hybrid retrieval to that structure, giving the parent graph most slots on a numbered log and balancing it with dense and lexical search elsewhere. It *assembles* the context in stored order, keeping the newest version of a numbered fact. An order the store does not record is worse than none. On fixed selections, shuffled serials score 59 on one fact-consolidation cell, below 64 with no order and 92 with the recorded one (Appendix H). Deterministic conflict resolution [3] assumes the serial from the task format and extracts facts with a model on every query, while MUSTER detects the serial and scores 93.0 and 60.0 on the fact-consolidation cells with the released implementation (92.0 and 37.0 in the original), against the 82.0 and 27.0 it reports at the same reader.

Contributions.

(i) **Detection before construction, for different kinds of agent memory.** MUSTER configures a store's read path from two closed-form statistics of its bytes, with no model, labels, per-store queries or candidate runs. It re-derives that configuration in seconds as the store grows. The share of numbered lines tells whether the store records an order and only then turns on keep-newest resolution and parent-priority retrieval. The share of text in short records tells whether to use a finer cut at record boundaries. The detected structure then sets how hybrid retrieval splits the slots among its channels, and assembly keeps stored order.

(ii) **First place on MemoryAgentBench.** The released implementation with the paper settings scores 60.5 against 43.6 at the benchmark's GPT-4o-mini reader and has the highest Overall among the retrieval and memory systems in Table 1 at that reader. It scores 62.1 and 67.0 with GPT-4.1-mini and GPT-5-mini, and 69.7 and 67.0 with GLM-5.3-Flash and GPT-6 Luna. Per-store automatic configuration scores 58.5 at GPT-4o-mini. Table 1 retains the original implementation for comparison.

(iii) **One rule, set on MemoryAgentBench, across three benchmarks.** On LoCoMo, the original implementation is the strongest row under both answer prompts (93.46 against 93.05 under the strongest system's own) with no generative build or search call, whereas that system makes 9,398. On LoCoMo-Refined it matches that system under the official evaluation (77.57 against 75.33) and leads it by 8.5 points with GLM-5.3-Flash. On BEAM, it matches the strongest system we compare at equal context from 100K to 10M tokens (71.3 against 70.8 for the strongest model-built memory). It has the highest mean with the open-weight Qwen3.5-9B at 9.7K tokens. It builds a 10M-token memory in 77 seconds instead of 13 hours and 10,429 calls.

(iv) **MUSTER works with or without a model-built memory.** The original implementation combines evidence from model-built memories and raw text (Section 5.4). We number Mem0's memories one per line in write order, so detection reads them as a numbered log,

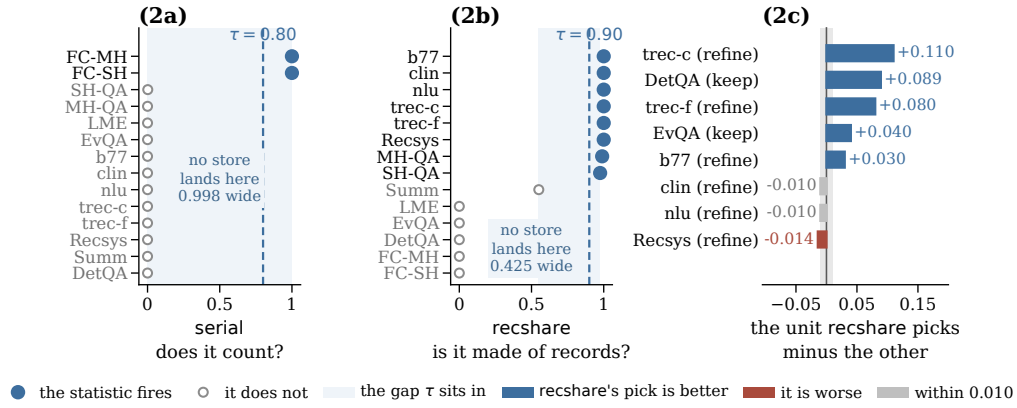


Figure 2: **Two statistics separate the stores. (2a, 2b)** Each statistic over the fourteen stores, filled where it fires (the rule checks serial first, so (2a) decides FC-SH and FC-MH), with τ inside a shaded interval no store occupies, 0.998 wide for serial and 0.425 for recshare. **(2c)** At $U = 10$ units, the unit recshare picks minus the other on the eight cells where both were run (Table 30). recshare's pick is better on five, within 0.010 on two, and worse on one (red).

and a team that runs Mem0 can keep its store. MUSTER reads them beside the conversation under one budget and averages 73.1 on BEAM, above either store alone (70.8 to 72.0) and every system at that budget.

2 RELATED WORK

Structure built at write time. RAPTOR [11], GraphRAG [12], HippoRAG [13, 14] and MemoRAG [15] spend a model per document before any query arrives, paying up front to scale to long histories. On MemoryAgentBench the strongest of them, HippoRAG-v2, ties a plain sparse retriever overall (41.6 against 41.5) and trails it by 8.7 on test-time learning (Table 1). When the store already records structure, reading it is cheaper, and checking which case holds costs seconds.

Memory systems compared. Mem0 [17], Honcho [31], Hindsight [33] and EverMemOS [36], which we run, write memory with a language model as each session arrives, at 4.4K to 26.1K language-model calls for one 10M-token conversation (Table 5), and MUSTER none. True Memory [30], which we also run, writes with no generative call and reads every store with one fixed representation. Mem0 2.1.0 also adds BM25 and entity scores to its dense ranking [32]. Table 1 further reports MemGPT [16], Zep [18] and MIRIX [19], and Section 5.1 compares Infini Memory [20] under its own scoring.

Memory without a generative write. Fidelity Before Structure [27] finds that verbatim chunks beat extracted artifacts and pools both in one list, whereas MUSTER gives each store its own share of the budget (Table 6). EdgeMem [28], like True Memory, builds memory with no generative call through one pipeline for every store, and LazyMem [29] defers construction to query time with a trained 4B model. MUSTER also writes with no generative call, trains nothing and adds detection, reading the configuration from each store's bytes.

Choosing the cut, resolving conflicts, assembling. Mix-of-Granularity [9], AutoRAG [26], HiChunk [10], FluxMem [37] and Adaptive Chunking [2] search for each store's RAG recipe by running every candidate, by labelled answers or by a trained model. OP-RAG [23], SeCom [24] and MacRAG [25] assemble by order or by parent over preset or model-cut units, and other 2026 work resolves conflicts once they are known [4, 5, 6, 7, 8, 22, 34]. MUSTER instead measures two byte statistics before retrieval and sets the unit, the parent and the order from them, including whether an order exists at all, and resolves conflicts as a read-time view (Appendix H).

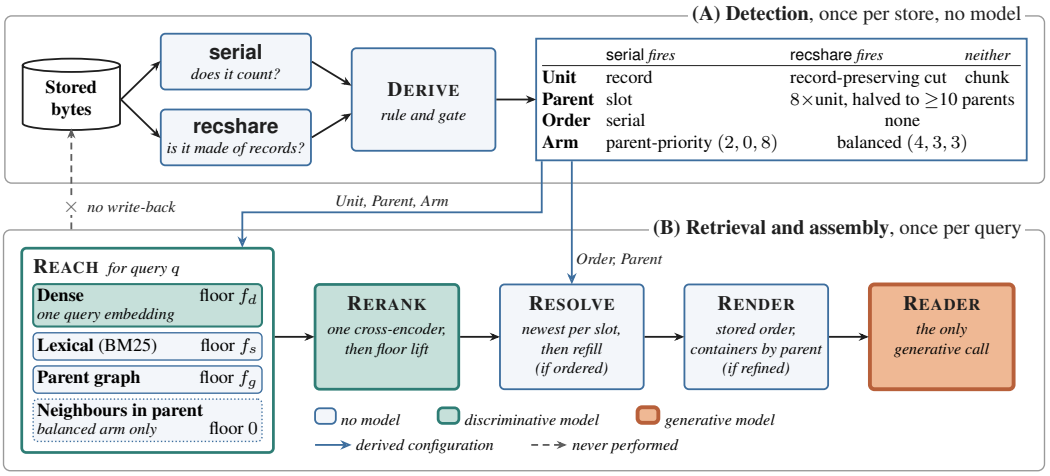


Figure 3: **MUSTER detects once, then reads** (Algorithm 1). **(A)** Detection derives unit, parent and order from two byte statistics and picks the retrieval arm once per plan, with no model. On MemoryAgentBench, the original implementation derives a representation for each cell from its first up to three stores. **(B)** Retrieval (reach, rerank) and assembly (resolve, render) answer each query with one query embedding, an initial rerank and one reader call over units embedded once per store. The SDK's numbered-fact walk can add rerank rounds. Nothing is written back. The refined branch, LoCoMo, LoCoMo-Refined and BEAM use one container per parent (Appendix A).

3 METHOD

MUSTER's three parts form one flow (Figure 3, Algorithm 1 in Appendix A). *Detection* sets unit, parent, order and channel floors from two byte statistics in seconds. *Structure-matched hybrid retrieval* reserves each channel its floor, most slots for the parent graph on a numbered log. *Context assembly* shows the selection in stored order, the newest version of each slot where an order is recorded. We call a retrieval policy an *arm*. Detection sees no dataset name, question or gold answer, so the rule set on MemoryAgentBench runs unchanged on LoCoMo, BEAM and a model-built memory.

3.1 DETECTION: BYTE STATISTICS

A retriever needs a unit, a parent and an order, and before the first query the only store content it may consult is the bytes. Two byte properties decide these choices. Whether an order exists is decided by whether the store counts. A leading integer on a line is the simplest surface form a writing process appends, and the rule takes a larger one as a later write. Whether the store may be cut finer depends on how much of its text lies in records that fit the reference chunk. Neither statistic needs a query, a label or a model, and recshare is measured against the cut the benchmark would otherwise make, which lets one threshold license a finer cut on a record store and refuse it on a novel.

$$\text{serial} = \frac{\#\{\ell \in L : \ell \text{ matches } \langle \textit{integer} \rangle \,.\, \langle \textit{body} \rangle\}}{|L|}, \qquad L = \text{non-empty lines}, \tag{1}$$

$$\text{recshare} = \frac{\sum_{r \in R,\, |r| \le c} |r|}{\sum_{r \in R} |r|}, \qquad R = \text{blank-line records},\ |r| = \text{tokens in } r, \tag{2}$$

where, in the original evaluation protocol, c is the ~~reference scale, the chunk sizethe benchmark prescribes for the cell and the one operating input the rule takes besides the bytes.~~ benchmark's reference chunk size. The current collection Auto API defaults to $c = 4096$. serial, short for *serial fraction*, asks whether the store counts. High means the store recorded its own order, and low means the rule claims no order to resolve by. Numbered lines do not push a conversation to the threshold. In the 65 BEAM conversations and the 500 LongMemEval-S histories [44] they are 15% to 35% of the lines on average and at most 45%, and no store reaches 0.80. recshare, short for *record share*, asks whether the store is made of records rather than merely containing some. With a high share, the cut follows record boundaries and slices records longer than the unit. With a low share, the store is narrative and a finer cut would be arbitrary. recshare weights records by their tokens. Counted

by records, InfBench-Sum would join the record stores, though two of its records hold 45.0% of its tokens (Appendix A).

3.2 DETECTION: DERIVATION RULE

The original evaluation follows this rule:

> If serial ≥ 0.80, the unit is the record with its serial stripped, the parent is the record's normalised slot, and the order is the serial. Otherwise the unit is the chunk size the benchmark prescribes. When recshare ≥ 0.90, the rule refines the unit by one documented step ($4096 \rightarrow 512$) and cuts at record boundaries, slicing only records longer than the resulting unit (the record-preserving cut). In the non-serial case, the parent is eight times the unit, halved until at least ten parents remain, and no order is claimed.

On MemoryAgentBench, the original implementation applies this rule to the first up to three stores of each cell and uses the resulting representation throughout that cell. ~~The released SDK 's paper-settings configuration preserves these automatically derived cell representations, while Auto derives a plan separately~~ SDK 0.1.3 exposes this scope through `muster.build_collection(stores)`: its defaults derive one plan from the first up to three stores, check the collection's source boundaries and build a separate memory for each store~~;~~. No benchmark name, query, gold answer or reference-chunk override is supplied.

Source boundaries also select the native recipe. Independent source records use the balanced arm with neighbours and flat rendering; dated speaker records and prose use the three channels and parent containers. A complete alternating session-date/turn-list literal preserves its user–answer rounds, unpaired non-empty turns and source sessions without a benchmark-specific adapter. Numbered logs retain their native read path. Appendix A gives the ~~channel and rendering choices~~input-reuse and historical evaluation conditions.

We follow DERIVE (Figure 3) end to end on two stores. FactConsolidation's store arrives as lines of the form `1. Alice's employer is Zynga`, so serial reads 0.9999. The unit becomes one record with the serial stripped, the parent a slot key naming the record's entity and attribute, read by a regular expression (Appendix A), and the order the integer itself. Nothing is chunked and nothing is embedded to decide it. The parent map is a real partition, so *resolve* has something to resolve by. DetectiveQA's store is a novel, where serial is 0 and recshare is 0, so the unit is the benchmark's own 4096-token chunk, the parent is eight of those halved until at least ten remain, and no order is claimed, which switches supersession off rather than letting keep-newest delete by an arbitrary key.

~~One~~ In the original benchmark protocol, one cell is outside the rule. The benchmark's loader delivers LongMemEval as conversation rounds grouped into sessions, and we keep that segmentation. The rounds are the units and the sessions the parents. Both statistics are still computed for it, and neither fires, but the rule's unit and parent are not applied. For the other thirteen cells, the rule determines the representation.

Besides the two thresholds, the rule's three constants come from the benchmark. They are the two chunk sizes it uses, 512 and 4096, their ratio of eight as the parent multiplier, and its retrieval budget of ten as the floor on parents (Table 9). Section 5 therefore tests a fixed rule.

3.3 DETECTION: POLICY GATE

A graph-heavy arm wins where the parent map carries meaning, and a dense-heavy one wins where it does not. The gate reads serial, the statistic that already chose the representation. A store where it fires takes the parent-priority arm, and every other store the balanced one (Section 3.4). serial's two classes are 0.998 apart, so any threshold in the gap gives the same partition. A model-based meaning test agrees with the gate on all fourteen stores (Appendix A).

3.4 STRUCTURE-MATCHED HYBRID RETRIEVAL: REACH AND CHANNEL FLOORS

Retrieval is a hybrid of three channels over the derived units (a dense embedding index, BM25, and a parent-level graph joining units that share an entity), and one cross-encoder scores whatever they

return. Ranked by one shared score, the pooled candidates let strong dense recall take almost every slot, and lexical or structural evidence that would answer the harder questions never reaches the reader. *Reach* therefore gives each channel a *reserved floor*, a minimum number of selected units drawn from its own ranking. The shared score fills the slots, and a channel left short trades the weakest selected unit outside it for its own next candidate.

At $U{=}10$ units the default arm, BALANCED, sets the floors $(f_d, f_s, f_g) = (4, 3, 3)$ and, in the paper configuration for MemoryAgentBench's record cells and LongMemEval, admits a fourth channel at floor zero, the immediate neighbours of each seed inside its own parent, which enters only on its score. PARENT-PRIORITY inverts the emphasis at $(2, 0, 8)$. Two dense anchors represent the query, and the parent graph spends the rest on breadth. Where the derived parent map has semantic meaning, that breadth is useful, and otherwise the balanced arm avoids paying for it (Table 2).

3.5 CONTEXT ASSEMBLY: READ-TIME SUPERSESSION AND CONTAINERS

Assembly shows the selection in stored order. Where an order was derived, *resolve* replaces each selected unit by the newest unit of its slot in the store, keeps one unit per slot and refills freed slots from the reranked dense candidates in rank order, without re-checking floors (Appendix A), so the context shows distinct current evidence rather than repeated versions of one slot. This is the *prune* view (Appendix I). Without a derived order, resolution is off. A wrong order can cost only units that share a slot (Appendix H).

Resolution happens at read time and is discarded with the context, so the store is never rewritten and a wrong call never deletes evidence from later queries. Write-time consolidation amortises its decision, and its error, over every later query.

The compensated refinement branch spans the three parts. Detection takes it for a 4096-token unit, retrieval searches at 1024 tokens and selects four times as many units, and assembly shows them one container per parent in stored order. The nominal allowance is unchanged at $40{,}960$ tokens, so the branch raises retrieval resolution rather than buying more reader context (Appendix A).

The released implementation. On dot-numbered stores, the SDK groups qualifying records by subject and relation, leaving values out of the slot key. A relation must occur for at least two subjects, with changing subjects averaging at most four records. Other records keep the original slot rule. The question seeds subjects whose current values name further subjects. The read path follows these links for at most four rounds, ranks each round and presents the selected chain in hop order. Across these rounds it reranks at most 400 walk candidates, in addition to the dense and parent-graph candidates already scored. Other numbering formats keep the original read path. SDK ~~(paper settings) and Auto use this same~~ 0.1.3 collection Auto retains this numbered-fact path from 0.1.2. Its FC rows reuse the archived native-path responses under the unchanged search and rendering conditions. ~~Their complete reader inputs on the 200 FC questions are identical, so the two configurations reuse the same responses and have the same FC scores.~~ Appendix A states the evaluation and development conditions.

4 SETUP

We evaluate all fourteen cells of MemoryAgentBench, an aggregate of separate public datasets whose stores span 25 units for a detective novel to $18{,}333$ for an assertion log, a range of $733\times$ that one threshold pair has to cross without per-dataset tuning (Table 8). We keep the benchmark's four competencies, accurate retrieval (AR), test-time learning (TTL), long-range understanding (LRU) and selective forgetting (SF), the last being the conflict-resolution family this paper turns on. Each cell keeps its own metric, substring match except GPT-4o judgment for LongMemEval and InfBench-Sum and recall@5 for Recsys. GPT-4o-mini [45], GPT-4.1-mini [40] and GPT-5-mini [50] each answer $1{,}671$ questions (Table 8), and Overall averages the four competency scores as [1] defines them. Mem0 2.1.0 *full-ver.* reads its top 100 extracted memories as the benchmark reads Mem0. Its raw-chunk row (the benchmark's chunks stored without extraction) and Honcho, Hindsight, EverMemOS and True Memory are cut at MUSTER's per-question budget, never below one whole unit, and on FactConsolidation each reads its own uncut retrieval (Appendix C).

Twelve cells are also rescored with Infini Memory's GPT-5 binary prompt [20], keeping Recall@5 and the summarisation judge. The two statistics and their thresholds were fixed before the rule's runs. The read-path settings (floor triples, view, K_1, encoder input length) were set on MemoryAgentBench and frozen for LoCoMo and BEAM (Appendix L).

5 RESULTS

In the results below, MUSTER is the strongest memory system or matches it at equal context on each benchmark. The ~~SDK rows report the current implementation on MemoryAgentBench~~collection Auto rows report SDK 0.1.3 on MemoryAgentBench; the per-store Auto rows retain version 0.1.2. The other benchmark results retain the original implementation. The rule is set on MemoryAgentBench, whose stores range from numbered logs to novels (Section 5.1). It runs unchanged on LoCoMo, LoCoMo-Refined, BEAM and a model-built memory (Sections 5.3 and 5.4).

5.1 MEMORYAGENTBENCH

~~The current implementation, released as library~~ SDK 0.1~~.2, scores 60.53, 62.14 and 67.02~~ .3 collection Auto scores 60.60, 62.18 and 67.04 Overall with GPT-4o-mini, GPT-4.1-mini and GPT-5-mini, and ~~69.67~~ 69.73 and 67.03 with GLM-5.3-Flash and GPT-6 Luna (Table 1). ~~These SDK (paper settings) rows retain the paper's cell configuration. With~~ For each reader, the LongMemEval cell combines 279 archived identical-input results with evaluation of 21 changed inputs after retaining unpaired source turns. The other thirteen cells retain the verified input-equivalent archived results. The historical 0.1.2 per-store ~~automatic configuration, the three GPT readers~~ Auto rows score 58.53, 60.60 and 64.74 ~~, changing Overall by +1.01, −0.31 and +0.03 points from the original implementation. Both configurations share the FC gains, while their non-FC plans can differ.~~ with the three GPT readers. Appendix A ~~gives the reader and answer-reuse~~ states the verification and reuse conditions.

The following comparisons use the original implementation.

First among retrieval and memory systems. At the benchmark's reader, GPT-4o-mini, MUSTER leads every retrieval and memory system in Table 1 on the total, 57.5 against 43.6 for the strongest, True Memory, 41.6 for HippoRAG-v2, which builds its graph with a language model, and 41.5 for BM25, which uses none. MUSTER also leads on three of the four competencies, and against True Memory on all four, widest on selective forgetting, where detection acts (64.5 against 26.0). At the same reader it beats the whole store read in context, by 15.3 at GPT-4o-mini, 14.0 at GPT-4.1-mini and 4.1 at GPT-5-mini, and with GPT-4o-mini it is also 8.7 above the stronger GPT-4o and 7.9 above Claude-3.7. Of the 15.9 over HippoRAG-v2, 8.8 is from SF and 7.1 from the other three.

Raw chunks above extraction, below MUSTER. Consistent with [27], Mem0 2.1.0 scores higher on the raw chunks than on the memories GPT-4o-mini or GPT-OSS-120B writes from them (38.9 against 26.7 and 28.6), mostly because both writers drop the labels of in-context examples, which leaves MCC at 5.6 and 12.0 against 67.4 on the raw chunks (Appendix C). The raw-chunk row is also a control for MUSTER. With the same text, reader and prompt and at least MUSTER's per-question budget, it scores 38.9 against MUSTER's 57.5 and is lower in every column. The two differ in all three parts (cut, retrieval and assembly). MUSTER sets all three from the store's bytes, with no per-store tuning.

Two competencies show detection most clearly. Conflict resolution is where serial fires and multi-hop is where it matters. The 37.0 there is the highest value at the benchmark's reader among the rows of Table 1 outside the SDK block (42.0 with GPT-4.1-mini and GPT-5-mini), above the 400K reader's 28.0, in a column where twenty-four of the twenty-five rows of the other systems are in single digits. The competency reads 64.5 against that reader's 53.0. In-context learning is where recshare licenses a finer cut, worth 4.0 on the MCC aggregate and 1.3 on test-time learning (both at $U = 10$, Table 30). On long-range understanding, MUSTER leads every retrieval and memory system on DetectiveQA, 66.2, above MIRIX's 62.0 at the stronger GPT-4.1-mini reader.

Refinement raises retrieval resolution without adding reader context. EventQA rises from 75.0, 74.0 and 70.0 with ten 4096-token units to 82.0 at all three readers when the coarse store is searched finer under the same nominal token allowance (Appendix J).

Table 1: **MemoryAgentBench with the current SDK and the original implementation.** MemoryAgentBench [1]. Bold: MUSTER. †Built with GPT-OSS-120B [49] in place of the model of the system's paper. MCC: mean of the five in-context-learning cells. TTL: mean of MCC and Rec. All: mean of the four competencies. Collection Auto is SDK ~~(paper settings) uses released library~~ 0.1~~.2 in native mode~~ .3 with ~~the paper's cell~~ default source-derived configuration. The historical per-store Auto rows use 0.1.2. Current rows combine input-equivalent archived results with the ~~same library, deriving each store's plan at the cell's protocol chunk size P and retaining~~ 21-input LongMemEval ~~'s rounds adapter~~refresh per reader (Appendix A). ~~Both use the updated FC path.~~ Original rows retain the original implementation.

	AR				TTL		LRU		SF (FC)		means				
Method	SH	MH	LME	EvQA	MCC	Rec	Summ	Det	SH	MH	AR	TTL	LRU	SF	**All**
Reader: GPT-4o-mini [45], the benchmark's reader															
***retrieval and write-time structure**: reported in [1]*															
BM25	66.0	56.0	45.3	74.6	75.4	13.6	19.0	52.1	48.0	3.0	60.5	44.5	35.6	25.5	41.5
Text-Emb-3-L	54.0	44.0	50.3	70.0	72.4	16.2	18.2	56.3	28.0	4.0	54.6	44.3	37.3	16.0	38.0
Qwen3-Emb-4B	57.0	47.0	43.3	71.4	78.0	12.2	14.8	59.2	29.0	3.0	54.7	45.1	37.0	16.0	38.2
RAPTOR [11]	29.0	38.0	34.3	45.8	59.4	12.3	13.4	42.3	14.0	1.0	36.8	35.9	27.9	7.5	27.0
GraphRAG [12]	47.0	47.0	35.0	34.4	39.8	9.8	0.4	39.4	14.0	2.0	40.9	24.8	19.9	8.0	23.4
MemoRAG [15]	29.0	33.0	20.0	56.0	77.0	13.1	9.2	50.7	21.0	7.0	34.5	45.1	30.0	14.0	30.9
HippoRAG-v2 [14]	76.0	66.0	50.7	67.6	61.4	10.2	14.6	57.7	54.0	5.0	65.1	35.8	36.2	29.5	41.6
***memory systems**: reported in [1]*															
Zep [18]	44.0	25.0	38.3	42.5	62.8	12.1	4.2	28.2	7.0	3.0	37.5	37.5	16.2	5.0	24.0
MemGPT [16]	41.0	38.0	32.0	26.2	67.6	14.0	2.5	42.3	28.0	3.0	34.3	40.8	22.4	15.5	28.3
MIRIX [19]	62.0	61.0	37.3	29.8	38.4	9.8	9.9	40.8	14.0	2.0	47.5	24.1	25.4	8.0	26.2
Mem0 [17] reported in [1]	25.0	32.0	36.0	37.5	32.4	10.0	4.8	36.6	18.0	2.0	32.6	21.2	20.7	10.0	21.1
***write- and read-time memory**: model-written memories, run here*															
Mem0 2.1.0, full-ver. [32]	48.0	47.0	49.7	55.0	5.6	12.3	19.1	59.2	15.0	3.0	49.9	9.0	39.1	9.0	26.7
at MUSTER's budget	46.0	51.0	50.0	52.0	5.6	12.8	20.0	54.9	15.0	3.0	49.8	9.2	37.5	9.0	26.4
Mem0 2.1.0, full-ver.†	59.0	57.0	53.0	61.0	12.0	14.8	15.2	58.7	8.0	5.0	57.5	13.4	36.9	6.5	28.6
Honcho [31]	40.0	47.0	49.7	50.0	35.0	12.5	35.1	56.3	41.0	7.0	46.7	23.8	45.7	24.0	35.0
Hindsight 0.10.3 [33]	79.0	73.0	42.3	53.0	6.4	12.3	13.1	43.7	46.0	6.0	61.8	9.4	28.4	26.0	31.4
EverMemOS† [36]	26.0	34.0	45.0	41.0	27.6	11.8	8.6	40.8	20.0	3.0	36.5	19.7	24.7	11.5	23.1
***read-time memory only**: one retrieval pipeline for every store, run here*															
Mem0 2.1.0, raw chunks	55.0	32.0	31.3	67.0	67.4	10.7	18.8	64.8	51.0	6.0	46.3	39.0	41.8	28.5	38.9
True Memory [30]	78.0	60.0	42.0	75.0	73.2	13.8	19.0	63.4	47.0	5.0	63.8	43.5	41.2	26.0	43.6
***full store**: long-context reader, oldest chunks evicted past the window, reported in [1]*															
GPT-4o-mini 128K	64.0	43.0	30.7	59.0	82.0	15.1	28.9	63.4	45.0	5.0	49.2	48.6	46.2	25.0	42.2
***read-time memory only**: structure read from the bytes, ours*															
Original, GPT-4o-mini	**92.0**	**65.0**	**56.3**	**82.0**	**81.8**	**13.3**	**22.2**	**66.2**	**92.0**	**37.0**	**73.8**	**47.5**	**44.2**	**64.5**	**57.5**
Stronger readers, named in each row, all but MUSTER reported in [1]															
MIRIX 4.1-mini [19]	73.0	75.0	51.0	53.0	61.0	10.3	18.9	62.0	20.0	3.0	63.0	35.7	40.5	11.5	37.7
***full store**: long-context readers, oldest chunks evicted past the window, reported in [1]*															
GPT-4o 128K	72.0	51.0	32.0	77.2	87.6	12.3	32.2	77.5	60.0	5.0	58.1	50.0	54.9	32.5	48.8
Claude-3.7 200K	77.0	53.0	34.0	74.6	89.4	18.3	52.5	71.8	43.0	2.0	59.7	53.9	62.2	22.5	49.6
GPT-4.1-mini 1M	83.0	66.0	55.7	82.6	75.6	16.7	41.9	56.3	36.0	5.0	71.8	46.2	49.1	20.5	46.9
GPT-5-mini 400K	85.0	71.0	63.3	78.2	84.0	13.2	56.3	76.1	78.0	28.0	74.4	48.6	66.2	53.0	60.6
Original, GPT-4.1-mini	**90.0**	**76.0**	**61.7**	**82.0**	**78.6**	**15.7**	**32.3**	**70.9**	**93.0**	**42.0**	**77.4**	**47.1**	**51.6**	**67.5**	**60.9**
Original, GPT-5-mini	**93.0**	**78.0**	**71.0**	**82.0**	**84.2**	**11.7**	**44.5**	**79.3**	**94.0**	**42.0**	**81.0**	**47.9**	**61.9**	**68.0**	**64.7**
~~**Current implementation: SDK (paper settings), library 0.1.2**~~**Current SDK 0.1.3: collection Auto**															
~~**SDK**~~**Auto, GPT-4o-mini**	**92.0**	**65.0**	~~**56.3**~~**57**	**83.0**	**81.8**	**13.3**	**21.9**	**66.2**	**93.0**	**60.0**	~~**74.1**~~**74**	**47.5**	**44.0**	**76.5**	~~**60.5**~~**60.6**
~~**SDK**~~**Auto, GPT-4.1-mini**	**90.0**	**76.0**	~~**61.7**~~**62**	**81.0**	**78.6**	**15.7**	**32.6**	**70.9**	**94.0**	**51.0**	~~**77.2**~~**77**	**47.1**	**51.8**	**72.5**	~~**62.1**~~**62.2**
~~**SDK**~~**Auto, GPT-5-mini**	**93.0**	**78.0**	~~**71.0**~~**71**	**82.0**	**84.2**	**12.2**	**44.5**	**79.3**	**95.0**	**59.0**	~~**81.0**~~**81**	**48.2**	**61.9**	**77.0**	**67.0**
~~**SDK**~~**Auto, GLM-5.3-Flash**	**97.0**	**77.0**	~~**72.0**~~**73**	**92.0**	**91.0**	**15.7**	**42.6**	**84.0**	**94.0**	**61.0**	~~**84.5**~~**84**	**53.3**	**63.3**	**77.5**	**69.7**
~~**SDK**~~**Auto, GPT-6 Luna**	**94.0**	**77.0**	**74.7**	**81.0**	**88.8**	**15.5**	**38.2**	**78.4**	**94.0**	**58.0**	**81.7**	**52.2**	**58.3**	**76.0**	**67.0**
~~**SDK: per-store automatic configuration**~~**Historical SDK 0.1.2: per-store Auto**															
Auto, GPT-4o-mini	**89.0**	**66.0**	**56.3**	**77.0**	**79.2**	**12.1**	**13.6**	**66.2**	**93.0**	**60.0**	**72.1**	**45.6**	**39.9**	**76.5**	**58.5**
Auto, GPT-4.1-mini	**90.0**	**77.0**	**61.7**	**83.0**	**79.2**	**17.3**	**16.7**	**70.9**	**94.0**	**51.0**	**77.9**	**48.2**	**43.8**	**72.5**	**60.6**
Auto, GPT-5-mini	**93.0**	**79.0**	**71.0**	**82.0**	**83.0**	**13.8**	**25.3**	**79.3**	**95.0**	**59.0**	**81.3**	**48.4**	**52.3**	**77.0**	**64.7**

Under Infini-style evaluation [20], MUSTER reaches 67.4 with GPT-5-mini against Infini Memory's reported 64.7 at the same reader, with gains in accurate retrieval (89.2 against 81.2) and selective forgetting (67.5 against 58.0), the two competencies the gate and resolution act on (Appendix B).

Table 2: **Detection and policy controls.** MemoryAgentBench [1], GPT-4o-mini [45] reader. The original-implementation rows give the 7.2-point detection comparison. The ~~separate~~ historical SDK 0.1.2 block compares parent-priority everywhere with the structure-matched gate on all fourteen cells under the paper settings. Bold: MUSTER.

Configuration	AR	TTL	LRU	SF	Overall
Hybrid retrieval and assembly, benchmark chunks	71.8	43.6	42.9	43.0	50.3
+ detection, BALANCED everywhere	73.8	47.5	44.2	58.5	56.0
+ detection, PARENT-PRIORITY everywhere	70.3	47.7	39.5	64.5	55.5
+ detection with its gate (MUSTER)	**73.8**	**47.5**	**44.2**	**64.5**	**57.5**
~~**Current SDK, paper settings**~~**Historical SDK 0.1.2, paper settings**					
SDK, PARENT-PRIORITY everywhere	70.5	47.7	39.4	76.5	58.5
SDK, structure-matched gate	**74.1**	**47.5**	**44.0**	**76.5**	**60.5**

5.2 ABLATIONS

With ~~the current SDK and~~ SDK 0.1.2 and the paper settings, the structure-matched gate scores 60.53 Overall against 58.51 for parent-priority retrieval on every cell, a 2.0-point advantage as in the original implementation (Table 2). Both retain the same cut, parents, read budget, rendering and retrieval models. Both SDK rows score 76.5 on selective forgetting, so their net difference comes from the other three competencies.

In the original implementation, each part has its own ablation. *Detection* adds 7.2 Overall to hybrid retrieval and context assembly over the benchmark's chunks under balanced floors, cut at MUSTER's per-question budget, from 50.3 to 57.5, and raises all four competencies (Table 2). With the balanced policy fixed, the units, parents and order it sets give 5.7 of the 7.2, and its policy gate the other 1.5. Byte-read slots match an entity-aware extractor with no model call (Appendix H). The gate adds 2.0 Overall over the parent-priority policy (Table 2). *Structure-matched hybrid retrieval* scores 6.0 higher on SF with its parent-priority arm than with its balanced arm (Table 2), and with assembly it adds 4.4 points per cell over the same detected units ranked by the cross-encoder alone, with no floors, parents or order (Table 29). *Context assembly* raises judged summarisation on all three readers with one container per parent at a fixed allowance, pooled $+0.036$ ($p = 0.017$, Appendix J). The unit count carries over to LoCoMo, where ten units, the budget MemoryAgentBench prescribes, score above five, twenty and forty under Mem0's answer prompt (Table 24).

5.3 TRANSFER TO LOCOMO

LoCoMo [35] holds ten two-person conversations and 1,540 questions in four answerable categories. The derivation rule of Section 3.2 applies unchanged. Each turn is one record (recshare 1.000, serial 0), so the rule takes 512-token units and halves the parent to 2,048 tokens so that at least ten parents remain, and we read $U{=}10$ units (5.0K tokens). LoCoMo prescribes no answer prompt, and the two in use move every store by 7.4 to 11.9 points, so each store is read under both, Mem0's [17] and EverMemOS's [36]. GPT-4.1-mini [40] is the reader and GPT-4o-mini [45] the judge. Every competitor runs its released code (Appendix F), and Table 3 lists each system's context and language-model calls.

Equal to the strongest system, with no generative model before the answer. Our EverMemOS run reproduces its reported 93.05, and MUSTER scores 93.46 under EverMemOS's answer prompt and leads every store under Mem0's, by 2.2 to 18.7 points. EverMemOS makes 9,398 language-model calls to build and search its memory, and MUSTER none. True Memory, which also builds memory with no generative model, stays 2.9 to 4.8 points below MUSTER with or without its HyDE query. MUSTER matches the whole conversation under EverMemOS's prompt and scores 4.0 higher under Mem0's, mostly on temporal questions (Appendix F), while reading a sixth of it.

Equal to the strongest system again, and ahead of it with a reasoning reader. LoCoMo-Refined [41] keeps LoCoMo's conversations, corrects its answers and judges each acceptable answer with Qwen3-14B [43]. Every store is rebuilt from its files, the model-built ones but Honcho's with GPT-OSS-120B, and every context is read under the answer template of the Agent Memory Leaderboard (AML) [42] (Appendix G). With GPT-4o-mini [45], MUSTER scores 77.57 against EverMemOS's 75.33 ($p = 0.06$), making no generative build or search call, whereas EverMemOS makes 9,034. MUSTER is 3.8 to 21.1 points above the other memory systems and 5.5 above the whole conversation

Table 3: **Highest memory system on LoCoMo.** LoCoMo [35] and LoCoMo-Refined [41], the same conversations with corrected answers, in LLM-judge accuracy. Readers: GPT-4.1-mini [40] on LoCoMo (judge GPT-4o-mini), and GPT-4o-mini [45], GLM-5.3-Flash [38] and Claude Sonnet 5.5 [47] on LoCoMo-Refined (its official judge, Qwen3-14B). Table 31 names the write models. Bold: MUSTER. [§]One HyDE query [48] per question. n/o, not observable.

	LoCoMo, 1,540 questions				**LoCoMo-Refined**, 1,382 questions				
	Answer prompt		Reader	LM	Official judge, by reader			Reader	LM
Store read, retrieval	Mem0	EverMemOS	context	calls	GPT-4o-mini	GLM-5.3-Flash	Sonnet 5.5	context	calls
Mem0 2.1.0 [32]	80.50	89.44	4.7K	5,882	73.73	80.54	83.50	4.9K	5,882
Honcho [31]	67.34	77.62	5.2K	n/o	56.44	65.56	68.09	5.2K	n/o
Hindsight 0.10.3 [33]	73.48	85.35	5.0K	602	63.68	70.91	73.08	4.6K	543
EverMemOS [36]	83.87	93.05	2.4K	9,398	75.33	80.32	83.86	4.0K	9,034
HippoRAG-v2 [14]	75.09	84.46	4.9K	2,073	67.58	76.05	76.85	4.9K	1,926
True Memory [30], HyDE	81.93	90.58	4.9K	1,540[§]	71.56	80.10	84.23	4.9K	1,382[§]
True Memory, no HyDE	81.19	90.35	4.9K	0	71.71	78.44	84.23	4.9K	0
Full store	82.06	93.44	31.1K	0	72.07	88.93	90.38	31.1K	0
MUSTER, U=10	**86.04**	**93.46**	**5.0K**	**0**	**77.57**	**88.78**	**89.58**	**5.0K**	**0**

Table 4: **Highest BEAM mean with Qwen3.5-9B.** BEAM [21] at MUSTER's 20 units (9.7K tokens), judged by GPT-4.1-mini, with the build cost of Table 5 (EverMemOS: all at once). Reader context at 1M. Honcho, Hindsight and EverMemOS read their own context. Write models: Table 31. Bold: MUSTER.

Reader: Qwen3.5-9B [39]	Reader	BEAM's answer prompt				Our answer prompt				LM calls	Build time
Store read, retrieval	context	100K	1M	10M	Mean	100K	1M	10M	Mean	10M conv.	100K/1M/10M
Mem0 2.1.0, full-ver., 9.7K [32]	9.7K	44.7	43.1	35.9	41.2	62.9	62.9	50.4	58.7	10,429	11m/77m/13h
Honcho [31]	20.7K	40.2	34.2	27.6	34.0	58.5	53.8	42.6	51.7	4.4–11.7K	7m/28m/7.8h
Hindsight 0.10.3 [33]	8.4K	40.4	40.5	34.2	38.4	60.2	59.9	48.2	56.1	16,425	49m/30m/3.0h
EverMemOS [36]	4.5K	44.2	43.6	37.5	41.8	61.9	62.6	55.1	59.9	26.1K	43m/7.7h/32.7h
HippoRAG-v2 [14]	9.7K	46.9	43.8	38.3	43.0	64.0	62.2	49.6	58.6	20.9K	1.0m/6.0m/1.3h
True Memory [30], 9.7K	9.7K	47.4	44.1	36.3	42.6	66.4	63.0	50.2	59.8	0	8s/3.3m/39m
MUSTER, U=20	**9.7K**	**48.7**	**43.9**	**39.9**	**44.2**	**65.3**	**62.5**	**53.1**	**60.3**	**0**	**1s/7s/77s**

($p < 0.001$). The gap widens with reasoning readers. With GLM-5.3-Flash [38], MUSTER scores 88.78, 8.5 points above EverMemOS and 8.2 or more above every memory system, and it answers more questions correctly than Mem0 and EverMemOS in every category (Appendix G). With Claude Sonnet 5.5 [47] and GPT-6 Luna [46] (Table 7), it stays at least 5.3 and 8.2 points above every memory system ($p < 0.001$, Tables 18 and 27) and matches the whole conversation.

5.4 TRANSFER TO BEAM

The rule runs on BEAM [21] unchanged (Appendix L). Detection reads each conversation as record-shaped (recshare 0.986) with too few numbered lines to order by (serial 0.18) and refines 4096 to 512-token units, while on the narrative cells of MemoryAgentBench it keeps the 4096-token chunk. Both choices score higher than the alternatives. On BEAM-100K the detected record units score above the other three cuts, by 0.6 to 17.3 points (Table 23), and the finer cut scores 4.0 and 8.9 points lower on EventQA and DetectiveQA (Table 30). These BEAM results use the original implementation, with $U = 20$ and 40 as resource-budget settings. We read $U = 20$ and 40 units (9.7K and 19.5K tokens) with GLM-5.3-Flash [38] and the open-weight Qwen3.5-9B [39], under BEAM's answer prompt and ours, judged by GPT-4.1-mini with BEAM's official code. Mem0 writes with GPT-4o-mini and Hindsight, EverMemOS and HippoRAG-v2 with GPT-OSS-120B (Table 31).

Above every memory system on the mean, with an open-weight reader. With Qwen3.5-9B, the only generative model MUSTER calls, MUSTER at 20 units has the highest mean under both answer prompts, 0.4 to 10.2 points above every memory system (Table 4), and under BEAM's prompt paired 95% intervals exclude zero against Mem0, Honcho, Hindsight and EverMemOS (Appendix C).

Parity at a fraction of the cost. With GLM-5.3-Flash (Table 5), MUSTER matches Mem0, the strongest model-built memory, at every scale and leads it at 1M on contradiction resolution and information extraction (Tables 14 and 15). It leads Honcho, Hindsight and EverMemOS at their

Table 5: **BEAM parity without a build model.** BEAM [21] score (ten-category macro of the official rubric judge with GPT-4.1-mini), reader context at 1M, and the cost of building memory for one 10M-token conversation. Build time: one conversation (EverMemOS: all at once). Bold: MUSTER. LM-written: tokens a language model wrote into the store. Honcho's calls are estimated. n/o, not observable.

Reader: GLM-5.3-Flash [38]	BEAM score, our prompt				Reader	Build cost, 10M conv.			Build time
Store read, retrieval	100K	1M	10M	Mean	context	LM calls	LM input	LM-written	100K/1M/10M
Mem0 2.1.0, full-ver. [32]	75.5	72.3	62.3	70.0	8.2K	10,429	≈101M	1.01M	11m/77m/13h
Mem0 2.1.0, full-ver., 19.5K	75.5	73.1	63.9	70.8	19.5K	10,429	≈101M	1.01M	11m/77m/13h
Honcho [31]	67.1	60.7	48.9	58.9	20.7K	4.4–11.7K	≥10.8M	n/o	7m/28m/7.8h
Hindsight 0.10.3 [33]	66.4	67.2	52.2	61.9	8.4K	16,425	65.5M	≈3.38M	49m/30m/3.0h
EverMemOS [36]	72.0	69.4	60.6	67.3	4.5K	26.1K	106M	3.85M	43m/7.7h/32.7h
EverMemOS, 19.5K	76.3	73.1	62.7	70.7	19.5K	26.1K	106M	3.85M	43m/7.7h/32.7h
HippoRAG-v2 [14], 9.7K	72.1	68.6	58.7	66.5	9.7K	20.9K	30.1M	1.35M	1.0m/6.0m/1.3h
HippoRAG-v2, 19.5K	75.7	71.3	60.0	69.0	19.5K	20.9K	30.1M	1.35M	1.0m/6.0m/1.3h
True Memory [30], 9.7K	75.4	72.1	61.4	69.6	9.7K	0	0	0	8s/3.3m/39m
True Memory, 19.5K	78.1	74.6	62.4	71.7	19.5K	0	0	0	8s/3.3m/39m
MUSTER, $U=20$	**75.0**	**72.4**	**62.7**	**70.0**	**9.7K**	**0**	**0**	**0**	**1s/7s/77s**
MUSTER, $U=40$	**76.1**	**73.4**	**64.3**	**71.3**	**19.5K**	**0**	**0**	**0**	**1s/7s/77s**

Table 6: **MUSTER beats Mem0's search on Mem0's memories, highest mean on both stores.** BEAM [21], judged by GPT-4.1-mini, with Mem0's model-written memories [32] read alone or beside the conversation at one reader budget. Bold: rows retrieved only by MUSTER. Δ: mean minus Mem0's search alone. –, none.

Reader: GLM-5.3-Flash [38]		Reader	BEAM score, our prompt				
Over the conversation	Over Mem0's memories [32]	context	100K	1M	10M	Mean	Δ
one store							
–	Mem0's search	19.5K	75.5	73.1	63.9	70.8	–
–	**MUSTER**	**19.5K**	**76.0**	**73.6**	**66.4**	**72.0**	**+1.1**
MUSTER, $U=40$	–	**19.5K**	**76.1**	**73.4**	**64.3**	**71.3**	**+0.4**
both stores, one budget							
MUSTER, $U=30$	Mem0's search	19.5K	77.3	73.3	64.2	71.6	+0.7
MUSTER, $U=30$	**MUSTER**	**19.5K**	**77.7**	**75.6**	**66.2**	**73.1**	**+2.3**

own contexts and HippoRAG-v2 at the same context, by 0.4 to 15.4 points, and True Memory is within 0.5 point on the mean and below MUSTER at 10M. Per 10M-token conversation, Mem0 makes 10,429 calls over 13 hours, while MUSTER builds in 77 seconds on one RTX PRO 6000 with no generative call (Table 16). Our Mem0 run scores 72.3 and 62.3 at 1M and 10M, above its published 63.1 and 54.8 under the same judge, so MUSTER faces a stronger Mem0 (Appendix C).

With a model-built memory, above either alone. MUSTER reads Mem0's memories, which we number one per line in write order (serial 1.0), under the same rule (Table 6), so a team that runs Mem0 can keep its store. Read after 30 units of the conversation under the same budget, they average 73.1, 2.3 points above Mem0's own search and 1.9 above MUSTER at 40 units (paired 95% intervals above zero, Appendix C). The two searches reach the same parents for 94% of the top ten memories at 100K but 61% at 10M, so at scale the memories reach evidence the conversation search misses. A fixed share for each store beats one pool (Appendix C).

5.5 ACROSS THE THREE BENCHMARKS

Table 7 and Figure 1 summarize the original implementation across the six settings. MUSTER ranks first overall and first or second in all six settings of Table 7. MUSTER's mean rank is 1.33 against 3.83 for the full store and its mean score 71.45 against 64.70 for True Memory (Figure 1). The full store falls to last on BEAM-10M, where the reader's window holds only the newest 850K tokens of each 11M-token conversation, while MUSTER reads 20K tokens per question and scores 28.4 to 30.0 points higher (Table 18). On MemoryAgentBench and LoCoMo-Refined, MUSTER is above every other memory system at both readers, including those that read as much or more ($p < 0.001$, Table 18), and on BEAM-10M it is level with Mem0, EverMemOS and True Memory, each reading 19K to 20K (Table 18). Each system reads the contexts of Section 5.1 and Tables 3 and 5, at MUSTER's per-question budget where it takes one and at its own default otherwise (Appendix D). MUSTER reads 5K to 20K tokens per question, at 1/34 of the full store's reader cost.

Table 7: **MUSTER ranks first with frontier readers.** Systems on MemoryAgentBench (MAB, All) [1], LoCoMo-Refined [41] and BEAM-10M [21], read by GLM-5.3-Flash [38] and GPT-6 Luna [46] and sorted by mean rank over the six settings. Bold: MUSTER. *Newest 850K tokens on BEAM-10M, the reader's window.

	MAB		LoCoMo-Refined		BEAM-10M		Mean	Mean	Worst	Top 3	LM calls
System	GLM	Luna	GLM	Luna	GLM	Luna	rank ↓	score ↑	rank ↓	↑	10M build
MUSTER	**67.70**	**64.50**	**88.78**	**87.84**	**64.30**	**55.55**	**1.33**	**71.45**	**2**	**6/6**	**0**
Full store*	64.46	61.30	88.93	87.41	34.27	27.14	3.83	60.59	8	4/6	0
True Memory [30]	58.27	56.06	80.10	78.44	62.45	52.91	4.33	64.70	5	0/6	0
Mem0 [32]	33.79	30.95	80.54	78.58	63.95	56.89	4.33	57.45	8	3/6	10,429
EverMemOS [36]	35.61	33.36	80.32	79.59	62.70	54.42	4.50	57.67	7	3/6	26,080
HippoRAG-v2 [14]	58.65	57.51	76.05	72.79	59.95	50.41	4.67	62.56	6	2/6	20,870
Hindsight [33]	38.62	37.67	70.91	69.10	52.19	46.28	6.33	52.46	7	0/6	16,425
Honcho [31]	41.73	38.81	65.56	62.16	48.89	44.64	6.67	50.30	8	0/6	4.4–11.7K

With ~~the paper settings, library~~ default collection Auto, SDK 0.1~~.2 scores 69.67~~ .3 scores 69.73 and 67.03 on MemoryAgentBench with GLM-5.3-Flash and GPT-6 Luna (Table 13); only the 21 changed LongMemEval inputs are evaluated anew for each reader.

6 CONCLUSION

RAG keeps every word and can score well after heavy tuning to a dataset, but the tuning does not generalize. Model-built memory scales to long histories but pays up front and can drop evidence. MUSTER finds the recipe in each store's structure, with neither per-store tuning nor a language model at write time. It detects that structure from the bytes, including whether an order exists, matches retrieval to it and assembles the context in stored order. Set on MemoryAgentBench, MUSTER ranks first among the retrieval and memory systems at the benchmark's reader. With the same derivation rule, it is the strongest row on LoCoMo under both answer prompts and matches the strongest system on LoCoMo-Refined, with no generative call before the answer. On BEAM it matches the strongest system we compare at equal context and builds a 10M-token memory in 77 seconds, where model-built memories take hours. By the same rule it also reads memories a language model has extracted, and on BEAM reading them beside the conversation gives a higher mean than either alone. Because MUSTER keeps each store as written and re-derives its configuration in seconds as the store grows, we expect it to serve the memory and long contexts of many AI agents.

AI USE STATEMENT

The authors conceived the research question, designed the MUSTER method and the experiments, directed the work and verified the results. Under their direction, generative AI tools assisted with writing and editing text and figures and with finding related work. In research execution, they assisted with data preparation, implementing the method, running experiments, and analysing and interpreting results. They also served as a sounding board for design choices, which the authors made. The AI tools created no data or benchmark content. The reported numbers come from the run artifacts and were checked by the authors. Language models used inside the experiments, such as readers, judges and memory writers, are part of the experimental setup that the paper describes. The authors take full responsibility for the paper.

REPRODUCIBILITY STATEMENT

The two statistics, serial and recshare, are given in closed form in Section 3.1, and the rule that consumes them is stated in full in Section 3.2. Neither needs a query, a label or a model call, so a reader can recompute both from the benchmark's released stores. Appendix A lists the retriever components, the reader and judge model identifiers, and the retrieval parameters needed to reproduce every cell. Appendix B reports all fourteen cells for all three readers under both MemoryAgentBench's metrics and Infini-style scoring, so that the MUSTER aggregates in Table 1 can be recomputed by hand. The harness makes no generative call beyond the reader per query. The harness, the per-question BEAM answer and judgment files and the scoring entry point are released with our code, and our answer prompt for BEAM is reproduced in full in Appendix C. MUSTER's DERIVE and build timings are on machines with one NVIDIA RTX PRO 6000 Blackwell Server Edition GPU. In the build only the embedding pass runs on the GPU, and the statistics, units, parent graph's entity index and BM25 run on the CPU. The embedding pass runs bge-m3 in fp16 for these timings and in fp32 for the reported MUSTER scores. Appendix C gives the setups behind other systems' build timings, in Table 16 for BEAM and Table 17 for MemoryAgentBench. Appendix F gives LoCoMo's prompts, judges and commits, and for LoCoMo-Refined Appendix G gives the benchmark's commit, the judge's model, revision and serving version, and every reader's settings.

ETHICS STATEMENT

This work evaluates retrieval over three public benchmarks, MemoryAgentBench, BEAM and LoCoMo, built from synthetic and public-domain text. LoCoMo-Refined, LoCoMo's relabelled release, is distributed under CC BY-NC 4.0. We use it for evaluation only and report aggregate results. This work involves no human subjects, no personal data, and no release of new data. The failure mode it reports, that an attribute-keyed extractor drops $2.7\times$ as many records as the byte-read slots, is a reliability concern for deployed memory systems and is reported so that it can be avoided.

REFERENCES

[1] Hu, Wang, and McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. ICLR 2026; arXiv:2507.05257v4.

[2] de Moura Júnior, Lelong, and Blangero. Adaptive Chunking: Optimizing Chunking-Method Selection for RAG. arXiv:2603.25333; LREC 2026.

[3] Reddy and Challaram. Reliable Post-Retrieval Assembly for Agent Memory: Separating Evidence Extraction from Policy Execution (v1: Don't Ask the LLM to Track Freshness: A Deterministic Recipe for Memory Conflict Resolution). Lifelong Agent Workshop at COLM 2026; arXiv:2606.01435v2.

[4] Patel. Supersede: Diagnosing and Training the Memory-Update Gap in LLM Agents. arXiv:2606.27472, 2026.

[5] Volkov, Li, and Luo. StateFuse: Deterministic Conflict-Preserving Memory for Multi-Agent Systems. arXiv:2607.05844, 2026.

[6] Zhou, Ouyang, Zheng, and Xiang. TEPA: Revoking Stale Memories for Conflict-Robust Language Agents. arXiv:2608.07429, 2026.

[7] Zhou et al. LatticeMind: A Conflict-Aware Memory Primitive for Multi-Agent Systems. arXiv:2608.08236, 2026.

[8] Wang. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv:2606.06240, 2026.

[9] Zhong, Liu, Cui, Zhang, and Qin. Mix-of-Granularity: Optimize the Chunking Granularity for Retrieval-Augmented Generation. COLING 2025; arXiv:2406.00456.

[10] Lu, Chen, Qiao, and Sun. HiChunk: Evaluating and Enhancing Retrieval-Augmented Generation with Hierarchical Chunking. arXiv:2509.11552, 2025.

[11] Sarthi et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024.

[12] Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.

[13] Jiménez Gutiérrez et al. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. NeurIPS 2024.

[14] Jiménez Gutiérrez et al. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. ICML 2025; arXiv:2502.14802; code at `https://github.com/OSU-NLP-Group/HippoRAG`.

[15] Qian et al. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. TheWebConf 2025; arXiv:2409.05591.

[16] Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[17] Chhikara et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.

[18] Rasmussen et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956, 2025.

[19] Wang and Chen. MIRIX: Multi-Agent Memory System for LLM-Based Agents. arXiv:2507.07957, 2025.

[20] Ji et al. Infini Memory: Maintainable Topic Documents for Long-Term LLM Agent Memory. arXiv:2606.10677, 2026.

[21] Tavakoli, Salemi, Ye, Abdalla, Zamani, and Mitchell. Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. ICLR 2026; arXiv:2510.27246.

[22] Yacoubi, Schmidt, Petrovic, Frikha, Kirchhoff, and Knoll. MemoryLACE: Memory Lifecycle-Aware Consolidation and Evidence Retrieval. arXiv:2609.03201, 2026.

[23] Yu, Xu, and Akkiraju. In Defense of RAG in the Era of Long-Context Language Models. arXiv:2409.01666, 2024.

[24] Pan et al. SeCom: On Memory Construction and Retrieval for Personalized Conversational Agents. ICLR 2025.

[25] Lim et al. MacRAG: Compress, Slice, and Scale-up for Multi-Scale Adaptive Context RAG. arXiv:2505.06569, 2025.

[26] Kim et al. AutoRAG: Automated Framework for optimization of Retrieval Augmented Generation Pipeline. arXiv:2410.20878, 2024.

[27] An. Fidelity Before Structure: Verbatim Chunks Beat Lossy Artifact Extraction in Long-Conversation LLM Memory. arXiv:2601.00821, 2026.

[28] Cui, Cao, Wen, Yuan, Feng, and Chen. EdgeMem: LLM-Free Agent Memory Construction and Retrieval via Evidence-Preserving Multi-Anchor Hypergraph. arXiv:2609.05553, 2026.

[29] Yu, Zhao, Zhang, and Li. LazyMem: Retrieve Broadly, Construct Selectively for Efficient Long-Term Agent Memory. arXiv:2607.22690, 2026.

[30] Adler and Zehavi. Storage Is Not Memory: A Retrieval-Centered Architecture for Agent Recall. arXiv:2605.04897, 2026; code at `https://github.com/buildingjoshbetter/TrueMemory`.

[31] Plastic Labs. Honcho, hosted service and open-source repository `https://github.com/plastic-labs/honcho` (ingestion runner at commit a1d689b; LoCoMo runner at 258fd37), accessed September and October 2026.

[32] Mem0 open-source release v2.1.0. `https://github.com/mem0ai/mem0`, accessed September 2026.

[33] Latimer, Boschi, Neeser, Bartholomew, Srivastava, Wang, and Ramakrishnan. Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. arXiv:2512.12818, 2025; code at `https://github.com/vectorize-io/hindsight`.

[34] Cui, Tang, Yao, Meng, Ma, and Jia. MemTxn: A Transaction Boundary for Source-Supported Updates and Complete-State Recovery in Agent Memory. arXiv:2607.27834, 2026.

[35] Maharana, Lee, Tulyakov, Bansal, Barbieri, and Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. ACL 2024; arXiv:2402.17753.

[36] Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing, and Deng. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. ACL 2026; arXiv:2601.02163; code at `https://github.com/EverMind-AI/EverMemOS`.

[37] Lu, Wu, Liu, Xu, Li, Wang, Hu, Ding, Sun, Lu, and Zhang. Choosing How to Remember: Adaptive Memory Structures for LLM Agents. arXiv:2602.14038, 2026.

[38] GLM-5 Team. GLM-5: from Vibe Coding to Agentic Engineering. arXiv:2602.15763, 2026; GLM-5.3-Flash weights at `https://huggingface.co/zai-org/GLM-5.3-Flash`.

[39] Qwen Team. Qwen3.5: Towards Native Multimodal Agents. `https://qwen.ai/blog?id=qwen3.5`, February 2026; Qwen3.5-9B weights at `https://huggingface.co/Qwen/Qwen3.5-9B`.

[40] OpenAI. Introducing GPT-4.1 in the API. `https://openai.com/index/gpt-4-1/`, April 2025.

[41] mem-eval-suite. LoCoMo-Refined. `https://github.com/mem-eval-suite/LoCoMo_refined`, commit 887091190789, CC BY-NC 4.0, accessed October 2026.

[42] Agent Memory Leaderboard. `https://agentmemoryleaderboard.ai`; public evaluation pipelines at `https://github.com/AML-memory/agent-memory-leaderboard`, accessed October 2026.

[43] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025.

[44] Wu, Wang, Yu, Zhang, Chang, and Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025; arXiv:2410.10813.

[45] OpenAI. GPT-4o mini: advancing cost-efficient intelligence. `https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/`, July 2024.

[46] OpenAI. GPT-6 Astra System Card, Appendix: GPT-6 Sol, GPT-6 Luna. https://deploymentsafety.openai.com/gpt-6-astra, September 2026.

[47] Anthropic. System Card: Claude Sonnet 5.5. https://www.anthropic.com/document/claude-sonnet-5-5-system-card, September 2026.

[48] Gao, Ma, Lin, and Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels. ACL 2023; arXiv:2212.10496.

[49] OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925, 2025.

[50] OpenAI. GPT-5 System Card. https://cdn.openai.com/gpt-5-system-card.pdf, August 2025; arXiv:2601.03267.

[51] Chen, Xiao, Zhang, Luo, Lian, and Liu. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. Findings of ACL 2024; arXiv:2402.03216.

[52] BAAI. bge-reranker-v2-m3. https://huggingface.co/BAAI/bge-reranker-v2-m3, accessed October 2026.

[53] Mem0. memory-benchmarks. https://github.com/mem0ai/memory-benchmarks, commit 4b61c5d, accessed September 2026.

A READ-PATH AND EVALUATION DETAILS

Algorithm 1 MUSTER. Part I runs detection, including the policy gate (step 11), once per plan with no model. On MemoryAgentBench, the original implementation derives a representation for each cell from its first up to three stores. Part II (retrieval and context assembly) uses one query embedding, an initial rerank and one reader call per query. The SDK's numbered-fact walk can add rerank rounds. $\tau_{\text{num}} = 0.80$ and $\tau_{\text{rec}} = 0.90$, and P is the benchmark's chunk (512 or 4096, Table 9). LongMemEval keeps its delivered rounds and sessions as units and parents (Section 3.2).

Input: store bytes S (never modified), query q. **Output:** the context.

1: **PART I, DERIVE** *(once per plan)*
2: serial ← share of lines opening with a serial marker; recshare ← share of text inside bounded records
3: **if** serial $\geq \tau_{\text{num}}$ **then**
4: unit ← record, parent ← slot, order ← serial
5: **else if** recshare $\geq \tau_{\text{rec}}$ **then**
6: unit ← record-preserving cut of $P/8$ tokens (at least 512)
7: **else**
8: unit ← sentence-bounded cut of P tokens; if $P > 512$, refine to $P/4$ and show $4\times$ the units
9: **end if**
10: **if** serial $< \tau_{\text{num}}$: parents ← $8\times$ the unit, halved until at least ten parents remain
11: channel floors ← parent-priority $(2, 0, 8)$ if serial $\geq \tau_{\text{num}}$, else balanced $(4, 3, 3)$
12: **PART II, READ** *(once per query)*
13: candidates ← dense ∪ lexical ∪ parent walk, each channel with a floor (when enabled, also each seed's neighbours in its parent, floor 0)
14: select the top units after the initial reranking pass; lift any channel below its floor
15: **if** order **then**
16: replace each unit by the newest unit of its slot in the store, keep one per slot and refill freed slots in rank order (a view, not a write)
17: **end if**
18: assemble in stored order, one container per parent when parent-grouped rendering is used; **return** the context

Table 8: **Store sizes vary 733-fold.** The fourteen MemoryAgentBench stores before compensated refinement, counted from the stores.

Cell	Source dataset	Units	Parents	n
DetectiveQA	DetectiveQA	25	12	71
InfBench-Sum	InfiniteBench	58	14	100
EventQA	EventQA	131	16	100
TREC-fine	TREC	202	25	100
CLINC150	CLINC150	206	25	100
TREC-coarse	TREC	207	25	100
NLU	NLU	208	26	100
Banking77	Banking77	210	26	100
SH-QA	RULER-style single-hop QA	398	49	100
LongMemEval	LongMemEval	719	89	300
MH-QA	RULER-style multi-hop QA	854	106	100
Recsys	ReDial	2,933	366	200
FC-SH	FactConsolidation	18,333	12,800	100
FC-MH	FactConsolidation	18,333	12,800	100

Table 8 counts units and parents from the stores themselves, not from the benchmark's description of them, so DetectiveQA, InfBench-Sum and EventQA show their 4096-token units before compensated refinement. The stores span $733\times$ in units and $1{,}067\times$ in parents, and one threshold pair crosses both ranges.

Under the serial branch of Section 3.2 the unit must be one record, and no chunk size delivers that. FactConsolidation's records span 7 to 30 tokens, so 16 splits the long ones and 32 glues the short ones. The branch therefore takes each record as one unit. At the 1.79 records per unit a chunk gives, the parent map and the order would both be meaningless.

Table 9: **Every constant is fixed across cells.** The constants Algorithm 1 uses without naming them. P: the cell's protocol chunk size, 512 or 4096.

Symbol	Value	What it decides	Where
Part I, derived once per store			
τ_{num}	0.80	serial numbering fires above this	§3.2
τ_{rec}	0.90	record-shaped rather than prose above this	§3.2
u	P or $P/8$	the read unit, from τ_{rec}	§3.2
$u_{\min}$	512	the finest unit the rule will cut to	§3.2
mult	$8, 4, 2$	parent layer, coarsest leaving ≥ 10 parents	§3.2
π	$(2, 0, 8)$ or $(4, 3, 3)$	channel floors, parent-priority or balanced	§3.4
U	10	units selected for the reader	§J.3
compensated refinement, when the store forced $u = P$			
u, U	$P/4,\ 40$	finer unit at the same token allowance	§3.5
α	$\max(1, \lfloor U/10 \rfloor)$	floors scale with the budget, $(4, 3, 3) \to (16, 12, 12)$	§3.4
Part II, per query			
R	parents	one container per parent in the selection	§3.5
K_1	200	first-stage candidates handed to the reranker	this section

In Eq. 2, c is the reference scale, P, and u the read unit. The token allowance is the same on both branches by construction, $4096{\times}10 = 1024{\times}40 = 40{,}960$ tokens, so the refinement buys resolution and not context.

Two properties of the reserved floors carry the design. A floor is filled from the channel's *own* ranking rather than credited to any candidate that also sits in that channel's top-K. Under the looser reading a dense unit lying deep in BM25's ordering satisfies the sparse floor by overlap and the lexical best never enters. Under BALANCED, floors are enforced after the global selection, largest floor first. The evicted unit is the lowest-scoring selected unit outside the channel whose removal keeps every other channel at its count, with ties broken by unit identifier. Candidate selection deduplicates unit identifiers, and the graph uses parents, but the balanced floors do not guarantee distinct parents.

Finer search, assembled context. Small units can retrieve a precise fact while omitting context needed to interpret it. A non-serial store that keeps a 4096-token unit under the rule is searched at 1024 tokens with $U = 40$ selected units. Floors scale by $\alpha = \max(1, \lfloor U/10 \rfloor)$ to $(16, 12, 12)$,

and the parent scale is recomputed by the same multiplier rule, giving 8192 tokens on the evaluated stores. This branch uses the three channels without the extra neighbour source and applies to EventQA, InfBench-Sum and DetectiveQA. LoCoMo, LoCoMo-Refined and BEAM also use the three channels without it. The other branches keep their policies, including parent-priority on serial stores. LongMemEval uses its native rounds and sessions rather than the token chunker (Section 3.2).

On the refined branch the forty selected units are shown one container per parent (8192 tokens on the evaluated stores), labelled `Memory i:`. A parent is never split across containers, and the containers and the units inside each follow stored order. LoCoMo, LoCoMo-Refined and BEAM show their selected units the same way, at their own unit counts and parent sizes. On the other branches of MemoryAgentBench the ten selected units are shown as they are, in stored order and without labels. The nominal content allowance remains $40 \cdot 1024 = 10 \cdot 4096 = 40{,}960$ tokens, while actual prompt length and execution cost can differ. Ten is the benchmark's `retrieve_num`, which its own prompt assembly turns into ten labelled containers for the sparse, dense and graph baselines alike. We fetch the same allowance but show as many containers as the selection has parents, the count Appendix J.3 measures. The alternative merges the forty units into at most ten containers ordered by rank, and Appendix J compares the two on MemoryAgentBench. The refined branch separates the resolution of retrieval from the context assembled for the reader.

A write-time extractor needs one generative call per unit, 6,151 across the twelve chunked stores alone, and the constructors in Table 1 spend one per document.

The dense encoder is BAAI/bge-m3 [51] in fp32, the sparse channel is BM25 and the cross-encoder is BAAI/bge-reranker-v2-m3 [52]. All runs set the first-stage parameter $K_1 = 200$, and the encoder and reranker read at most 2048 tokens of a MUSTER unit (shorter units are unaffected). The first row of Table 2 reads each 4,096-token chunk whole, so its control sees every chunk in full. Reader identifiers are `gpt-4o-mini-2024-07-18`, `gpt-4.1-mini-2025-04-14` and `gpt-5-mini`. The first two answer at temperature 0.7 with seed 42, the harness default. The third, `gpt-5-mini`, is a reasoning model that runs at its default sampling with no output cap, and its answer is scored in full. The Infini-style binary accuracy judge returned `gpt-5-2025-08-07`. Reusing a judgment requires equality of the full question, reference, prediction and request settings. Cell identity is read from each result file's recorded question type and parquet row, never from its filename, and scoring runs through one entry point that owns the metric map, so a cell can neither be scored by the wrong metric nor filled from a run it did not come from.

Rules of the original implementation. The serial pattern is `^\s*(\d+)\.\s+(.+)$`. On a serial store the slot key is the record's first capitalised entity span, lower-cased, joined to the record body in which every capitalised span is masked to _. Appendix J reports three coarser keys. Entities everywhere are capitalised spans found by a regular expression, minus hubs, the entities that appear in more than 5% of units. The graph channel represents each parent by one unit, drops entities shared by more than 5% of parents, seeds the parents that hold a query entity with that entity's IDF, walks a bounded number of hops with weight decay 0.5 per hop, orders the reached parents by hop and then accumulated IDF with ties by stored order, offers the first 100, and scores their representatives with the shared cross-encoder. On a serial store the walk runs over slots. *Resolve* replaces each selected unit by the largest-order unit of its parent in the store and keeps one unit per parent. It then refills from the reranked dense candidates in order, replacing each the same way and skipping parents already shown, without re-checking floors. On a serial store each kept unit is shown as `[serial] text` in serial order.

Why recshare weights records by their tokens. Counted by records, InfBench-Sum, measured on its first three novels, would join the record stores on 4,539 records, though two of them hold 45.0% of its tokens. Weighted, the fourteen stores split eight at 0.9751 or above and six at 0.5504 or below. Computing both statistics and applying the rule to all fourteen stores takes 2.8 seconds on a CPU after the stores are read (mean of three runs).

The gate's meaning test. A meaning test, asking GPT-4o-mini whether the competing values of sampled slots are mutually exclusive, agrees with the form test of Section 3.3 on all fourteen stores, with its classes 0.147 apart. The test measures InfBench-Sum on its first three novels and costs $0.12 in all. The partition thus holds at two levels of description, and only the form test is free.

Floors in priority order. Under PARENT-PRIORITY the two dense anchors are filled first and the parent graph then spends the remaining budget on breadth. A neighbour under BALANCED is never reserved a slot.

The released library's native path. On dot-numbered stores, ~~version~~ the path introduced in 0.1~~.2~~.1 and retained in 0.1.3 forms recognised subject and relation slots separately from their values. From question matches it follows value-to-subject links and orders the selected records by hop. For these FC rows it selects ten units. At most four walk rounds and 400 walk candidates are a separate search budget in addition to the initial retrieval. Other numbering formats retain the original read path. Slots are matched by text pattern. These rules were developed using the 600 FactConsolidation questions at the smaller scales and the reported 262K stores.

Library evaluation conditions. ~~SDK (paper settings)~~ Collection Auto in Tables 1, 10, 11, 12 and 13 ~~is the released implementation in native mode with the paper's cell configuration held fixed. That configuration preserves the cell representations originally derived from the~~ uses the SDK 0.1.3 default collection strategy, with no benchmark name, query, answer, reference-chunk override or explicit LongMemEval adapter. The first up to three ~~stores per cell. Auto uses per-store derivation in the same SDK at the cell's protocol chunk size P and retains LongMemEval's rounds adapter. The original record branches use the balanced arm with~~ source stores supply the shared statistics; each memory is cut separately. Independent records use neighbours and flat rendering~~on MemoryAgentBench, but the three channels without neighbours~~ , while dated conversation records and prose use three channels and parent-grouped rendering~~on LoCoMo and BEAM. On its record and prose branches, Auto uses neighbours and parent-grouped rendering. The two detection statistics therefore do not specify every channel and rendering choice across benchmarks. On MemoryAgentBench, relative~~ . Only a complete alternating session-date/turn-list literal activates the rounds path. The encoder and reranker retain the 2048-token input cap.

The default API was checked on all 1,471 non-FC questions against the archived SDK 0.1.2 paper-settings inputs. Complete reader messages matched for 1,450; 21 LongMemEval messages changed because the native cut now retains unpaired source turns. Those inputs were read under the original requested model, provider and request contracts. A changed prediction was judged with the original scorer protocol; an archived judgment was reused only for an identical complete judge payload. The archived actual judge snapshot was not recorded, so the same requested alias does not establish snapshot identity. Each reader has all 300 LongMemEval judgments. For the 200 FC questions, fixed selection traces were rendered again and the numbered search functions, cut fields and source data were checked unchanged; this was not a new neural FC retrieval run. CPU and bounded local GPU cache preparation are recorded separately; the final input replay used caches only. Input replay itself introduces no answer-quality measurement. DetectiveQA retains its three reads per question.

The historical per-store Auto rows and the SDK policy ablation in Table 2 retain version 0.1.2. Relative to the paper configuration, ~~Auto groups the retrieved units of~~ that per-store Auto groups SH-QA, MH-QA, the five in-context-learning cells and Recsys by parent, ~~under one `Memory n:` header per parent, and enables the neighbour channel~~ enables neighbours on EventQA, InfBench-Sum and DetectiveQA~~. Auto chooses~~ , and changes InfBench-Sum's branch ~~per novel, changing it~~ on 74 of ~~the~~ 100 novels. ~~Both paths use the original questions, reference answers, reader settings and MemoryAgentBench rubrics, with the encoder and reranker capped at 2048 input tokens. We read anew when no archived answer matches the full messages and request settings. Judgments are reused only after matching the full judge payload. On FC, the two configurations have identical complete inputs and reuse the same updated-path responses. DetectiveQA retains three reads per question.~~ The original LoCoMo and BEAM runs retain their three-channel, parent-grouped recipe and stated read budgets. The Infini-style columns ~~keep~~ retain the original implementation's values. The ~~SDK (paper settings) columns of Table 13 use this native path with each reader's original~~ frontier readers retain their high-effort and empty-answer fallback ~~contract. Ancillary~~ contracts. The source includes the archived per-cell CSVs ~~accompany the source~~and a collection-Auto CSV, the input-replay inventory and the question-level LongMemEval refresh receipt.

B COMPLETE CELL RESULTS

Tables 10, 11 and 12 give every cell ~~of MUSTER at the three readers,~~ at GPT-4o-mini, GPT-4.1-mini and GPT-5-mini. ~~In these three tables, the last column is MUSTER under~~ The SDK 0.1.3 Auto columns refresh 21 LongMemEval inputs per reader and otherwise reuse input-equivalent paper-settings results; the per-store Auto columns retain SDK 0.1.2. The last column uses Infini-style scoring, ~~where Rec and Summ keep their MemoryAgentBench metrics~~retaining Recall@5 and the summarisation judge. DetectiveQA is the mean of three reads. ~~SDK (paper settings) uses the current released implementation with the paper configuration. Auto uses per-store automatic configuration at all three readers.~~ Table 13 adds GLM-5.3-Flash and GPT-6 Luna~~under their original MemoryAgentBench reader and scoring contracts. For each of these two readers,~~. Their archived SDK 0.1.2 paper-settings runs read 224 changed-input response slots ~~are read anew and~~ anew and reused 1,589 identical-input slots~~reuse archived answers~~, including the three DetectiveQA reads. The current columns retain those responses except for the 21 LongMemEval inputs refreshed for each reader.

Table 10: **Every cell with GPT-4o-mini.** The fourteen MemoryAgentBench cells of Table 8, 0–100 scale. Original~~implementation~~: the original reported ~~results~~implementation. SDK ~~(paper settings): released library 0.1.2 in native mode~~ .3 Auto: default collection configuration, with the ~~paper's cell configuration~~LongMemEval refresh and verified archived results for the other cells. Per-store Auto: ~~per-store derivation at~~ the ~~cell's protocol chunk size P, with LongMemEval's rounds adapter~~historical 0.1.2 configuration. Infini-style retains the original ~~implementation's~~ judgments. Overall: mean of ~~the~~ four competencies~~(Table~~. Reader and reuse conditions are in Appendix ~~1)~~A. Bold: MUSTER.

Cell	n	Original implementation	~~SDK~~SDK 0.1.3 ~~(paper settings)~~Auto	0.1.2 ~~Auto~~ per-store Auto	MUSTER, Infini-style
SH-QA	100	**92.00**	**92.00**	**89.00**	**98.00**
MH-QA	100	**65.00**	**65.00**	**66.00**	**80.00**
LongMemEval	300	**56.33**	~~56.33~~**57.33**	**56.33**	**61.00**
EventQA	100	**82.00**	**83.00**	**77.00**	**85.00**
Banking77	100	**96.00**	**96.00**	**94.00**	**96.00**
TREC-coarse	100	**70.00**	**70.00**	**66.00**	**70.00**
CLINC150	100	**90.00**	**90.00**	**89.00**	**90.00**
NLU	100	**88.00**	**88.00**	**87.00**	**87.00**
TREC-fine	100	**65.00**	**65.00**	**60.00**	**65.00**
Recsys	200	**13.25**	**13.25**	**12.08**	**13.25**
InfBench-Sum	100	**22.24**	**21.86**	**13.58**	**22.24**
DetectiveQA	71	**66.20**	**66.20**	**66.20**	**76.06**
FC-SH	100	**92.00**	**93.00**	**93.00**	**92.00**
FC-MH	100	**37.00**	**60.00**	**60.00**	**37.00**
Overall		**57.52**	~~60.53~~**60.60**	**58.53**	**60.52**

For the Infini-style comparison we keep Recall@5 on Recsys and the GPT-4o Fluency×F1 score on InfBench-Sum rather than re-judging them, and Infini Memory's number is taken from its paper, which also reads with GPT-5-mini.

The original implementation's answers are scored under both protocols. All three readers sample (GPT-4o-mini and GPT-4.1-mini at temperature 0.7, GPT-5-mini at its default) and move DetectiveQA's 71 answers between runs of one context, so DetectiveQA is the mean of three reads in both original columns. The reads are 67.61, 66.20 and 64.79 with GPT-4o-mini, 70.42, 71.83 and 70.42 with GPT-4.1-mini, and 78.87, 83.10 and 76.06 with GPT-5-mini.

Table 11: **Every cell with GPT-4.1-mini.** The fourteen MemoryAgentBench cells of Table 8, 0–100 scale. Columns, Infini-style and Overall: as in Table 10. Bold: MUSTER.

Cell	n	Original implementation	~~SDK~~SDK 0.1.3 ~~(paper settings)~~Auto	0.1.2 ~~Auto~~ per-store Auto	MUSTER, Infini-style
SH-QA	100	**90.00**	**90.00**	**90.00**	**96.00**
MH-QA	100	**76.00**	**76.00**	**77.00**	**86.00**
LongMemEval	300	**61.67**	**~~61.67~~62.33**	**61.67**	**70.00**
EventQA	100	**82.00**	**81.00**	**83.00**	**88.00**
Banking77	100	**97.00**	**97.00**	**96.00**	**96.00**
TREC-coarse	100	**73.00**	**73.00**	**69.00**	**73.00**
CLINC150	100	**89.00**	**89.00**	**92.00**	**89.00**
NLU	100	**85.00**	**85.00**	**86.00**	**85.00**
TREC-fine	100	**49.00**	**49.00**	**53.00**	**47.00**
Recsys	200	**15.67**	**15.67**	**17.25**	**15.67**
InfBench-Sum	100	**32.33**	**32.62**	**16.65**	**32.33**
DetectiveQA	71	**70.89**	**70.89**	**70.89**	**74.65**
FC-SH	100	**93.00**	**94.00**	**94.00**	**92.00**
FC-MH	100	**42.00**	**51.00**	**51.00**	**45.00**
Overall		**60.92**	**~~62.14~~62.18**	**60.60**	**63.46**

Table 12: **Every cell with GPT-5-mini.** The fourteen MemoryAgentBench cells of Table 8, 0–100 scale. Columns, Infini-style and Overall: as in Table 10. Bold: MUSTER.

Cell	n	Original implementation	~~SDK~~SDK 0.1.3 ~~(paper settings)~~Auto	0.1.2 ~~Auto~~ per-store Auto	MUSTER, Infini-style
SH-QA	100	**93.00**	**93.00**	**93.00**	**97.00**
MH-QA	100	**78.00**	**78.00**	**79.00**	**93.00**
LongMemEval	300	**71.00**	**~~71.00~~71.33**	**71.00**	**78.67**
EventQA	100	**82.00**	**82.00**	**82.00**	**88.00**
Banking77	100	**95.00**	**95.00**	**93.00**	**95.00**
TREC-coarse	100	**84.00**	**84.00**	**82.00**	**84.00**
CLINC150	100	**94.00**	**94.00**	**94.00**	**94.00**
NLU	100	**89.00**	**89.00**	**85.00**	**88.00**
TREC-fine	100	**59.00**	**59.00**	**61.00**	**58.00**
Recsys	200	**11.67**	**12.17**	**13.83**	**11.67**
InfBench-Sum	100	**44.47**	**44.48**	**25.28**	**44.47**
DetectiveQA	71	**79.34**	**79.34**	**79.34**	**85.92**
FC-SH	100	**94.00**	**95.00**	**95.00**	**92.00**
FC-MH	100	**42.00**	**59.00**	**59.00**	**43.00**
Overall		**64.71**	**~~67.02~~67.04**	**64.74**	**67.40**

Table 13: **Every cell with GLM-5.3-Flash and GPT-6 Luna.** The fourteen MemoryAgentBench cells of Table 8, 0–100 scale. Original~~implementation~~: the MAB rows of Table 7. SDK ~~(paper settings): released library 0.1.2 in native mode~~ .3 Auto: default collection configuration with the ~~paper's cell configuration~~LongMemEval refresh and input-equivalent archived results for the other cells. The original reader, scoring and empty-answer fallback contracts are retained (Appendix A). DetectiveQA averages three reads~~.~~; Overall averages ~~the~~ four competencies.

		GLM-5.3-Flash		GPT-6 Luna	
Cell	n	Original implementation	~~SDK~~SDK 0.1.3 ~~(paper settings)~~Auto	Original implementation	~~SDK~~SDK 0.1.3 ~~(paper settings)~~Auto
SH-QA	100	**97.00**	**97.00**	**94.00**	**94.00**
MH-QA	100	**77.00**	**77.00**	**77.00**	**77.00**
LongMemEval	300	**72.00**	**~~72.00~~73.00**	**74.67**	**74.67**
EventQA	100	**93.00**	**92.00**	**80.00**	**81.00**
Banking77	100	**97.00**	**97.00**	**96.00**	**96.00**
TREC-coarse	100	**92.00**	**92.00**	**92.00**	**92.00**
CLINC150	100	**96.00**	**96.00**	**98.00**	**98.00**
NLU	100	**90.00**	**90.00**	**89.00**	**89.00**
TREC-fine	100	**80.00**	**80.00**	**69.00**	**69.00**
Recsys	200	**16.17**	**15.67**	**15.50**	**15.50**
InfBench-Sum	100	**42.92**	**42.62**	**37.45**	**38.19**
DetectiveQA	71	**84.04**	**84.04**	**78.40**	**78.40**
FC-SH	100	**93.00**	**94.00**	**92.00**	**94.00**
FC-MH	100	**45.00**	**61.00**	**41.00**	**58.00**
Overall		**67.70**	**~~69.67~~69.73**	**64.50**	**67.03**

C BEAM ACROSS SCALES: INSTRUMENT AND PROMPT

Instrument. GLM-5.3-Flash answers at maximum reasoning effort through one pinned provider. The judge is BEAM's official code, unmodified, with GPT-4.1-mini, and the score is the benchmark's ten-category macro average, aggregated question → conversation → category. Intervals come from a paired bootstrap over whole conversations with 10,000 draws. The questions are 400 over 20 conversations at 100K, 700 over 35 at 1M and 200 over 10 at 10M. Each message is written as one blank-line record holding the fields BEAM gives it (its id, date, session and turn index, speaker and text), and nothing else is added. The rule is read once per scale (pooled recshare 0.986 at 100K, 0.993 at 1M and 10M). At every scale an answer left empty was re-asked with an output allowance of 31,200 tokens (30,000 for reasoning) instead of 9,200. Over the runs reported here, 149 of 23,200 GLM-5.3-Flash answers were re-asked at 100K, 413 of 32,200 at 1M and 232 of 9,200 at 10M (at most 16, 48 and 29 in one run), and 6, 7 and 10 of them stayed empty because reasoning used the larger allowance too. Qwen3.5-9B, read without reasoning, left no answer empty. Means and Δ in Tables 4, 5 and 6 are of unrounded scores, and the reader context of Table 4, like that of Table 5, is measured at 1M. True Memory's ranked results are cut to MUSTER's two contexts, and its rows and MUSTER's in Table 5 average two reads.

Competitors. Mem0 is the open-source release 2.1.0 [32], run as a local server with the benchmark runner of its authors [53], returning the top 200 memories with their dates. Its ingestion and search differ from the system described in [17], which the Mem0 row of Table 1 reports as MemoryAgentBench measured it. The BEAM rows and the four MemoryAgentBench rows of release 2.1.0 are results of that release under this configuration. On MemoryAgentBench each 4,096-token chunk of the benchmark is one `add` call, wrapped in the benchmark's memorize template for extraction or stored bare with `infer=False` for the raw-chunk row, and the six chunks longer than the embedder's input are split in equal parts. Honcho receives the same chunks, each as one user message followed by a short acknowledgement, is read after its ingestion queue has drained, with its session summary requested, and runs with dreaming off. EverMemOS runs its released code at `fe436d3` on the same chunks and acknowledgements, one conversation per source text, with GPT-OSS-120B (reasoning effort low) answering every language-model call of construction and retrieval and Qwen3-Embedding-4B and Qwen3-Reranker-4B serving retrieval, all on vLLM on one RTX PRO 6000 (the three retrieval calls, capped at 500, 150 and 300 tokens, get 1,024 more for the reasoning). One GPT-OSS-120B answer needed a parse-only repair, which fixes only the format of the answer's JSON and changes no prompt, model or retry. EverMemOS's 8,192-token segment limit closes almost every MemCell at a chunk boundary (5,733 MemCells for 5,956 chunks), and the context its answer stage assembles is cut at MUSTER's per-question budget. Hindsight 0.10.3 retains each chunk as one item, one memory bank per source text in the text's order, with GPT-OSS-120B (reasoning effort low) on vLLM on one RTX PRO 6000 (227,840 facts from the 5,956 chunks), and its recall is cut at MUSTER's per-question budget. Mem0 2.1.0 full-ver. is read as the benchmark reads Mem0, with its top 100 memories and no token cut (1.0K to 4.9K tokens per cell with GPT-4o-mini writing and 0.06K to 15.5K with GPT-OSS-120B). Filled in its rank order to MUSTER's per-question budget from its top 1,000 instead, its whole store on the three novel cells, it scores 26.4. The other competitor contexts are cut at MUSTER's per-question budget, on average 4.3K tokens on SH-QA and MH-QA, 4.4K on Recsys, 5.7K on LongMemEval, 5.3K to 5.5K on the five in-context-learning cells, 40.3K to 40.5K on EventQA, InfBench-Sum and DetectiveQA, and never below one whole unit of the system. On the two FactConsolidation cells that budget (0.15K) is smaller than one raw chunk, so each system reads its own uncut retrieval, on average 42.4K tokens for Mem0's top ten raw chunks, 32.3K for Honcho's context with no token limit, 11.6K for Hindsight's default recall and 1.7K for EverMemOS's full context. Raw chunks are the benchmark's 4,096-token units, so a budget of 4.3K to 5.7K tokens holds one chunk, shown whole where it exceeds the budget. They are ranked by Mem0's search, which adds BM25 to the dense similarity of its embedder (text-embedding-3-small) and calls no reranker, and read by MUSTER's reader and prompt. With `infer=False` no entities are linked, so the search's entity-match score is inactive. Mem0's default similarity threshold leaves some contexts empty, and they are read as such. On TREC-coarse and TREC-fine 10 and 14 contexts are empty for raw chunks, 1 and 1 with GPT-4o-mini writing, and 49 and 15 (and 3 on FC-SH) with GPT-OSS-120B writing. In the contexts returned for the 500 in-context-learning questions, GPT-4o-mini keeps a label in 0 of 36,449 memory lines and GPT-OSS-120B in 945 of 21,383 (4.4%), and more than 99% of the lines of either writer open with "User". None of the 200 FactConsolidation contexts of either writer keeps a serial. Honcho is the hosted service [31], called through its

session-context API with the ingestion runner of its repository, after its ingestion queue had drained. A hosted service is not version-pinned, so the BEAM contexts it returned are released with our code, joined to their questions. past.dev (`https://past.dev/benchmarks/beam`, accessed September 2026) is the hosted service at its lowest recall level. It scores 72.3 through its API under our instrument against 92.1 on its site. Exabase M-1 (`https://exabase.io/research/exabase-achieves-state-of-the-art-on-beam-benchmark`, accessed October 2026) scores 42.6 under our instrument (67.0 with its inference off) against 76.9 on its site. Each system received every message with its BEAM timestamp, and each returned context was passed to the same reader with the same prompt.

What Mem0's writers keep on MemoryAgentBench. Both writers, GPT-4o-mini and GPT-OSS-120B, restate the labelled examples of MemoryAgentBench as statements about the user and keep few labels, GPT-4o-mini none, so in-context learning carries most of the gap to the raw chunks (MCC 67.4 against 5.6 and 12.0). Elsewhere extraction leads on accurate retrieval (49.9 and 57.5 against 46.3), and neither writer keeps the serials MUSTER resolves conflicts by. The raw-chunk row reads Mem0's uncut top ten on FactConsolidation, at least MUSTER's per-question budget, and differs from MUSTER in the cut (4,096-token chunks against detected units and parents), the retrieval (dense and BM25 ranking against structure-matched floors, a parent channel and a reranker) and the assembly (a ranked list against stored order with supersession).

Our answer prompt (system message). The user message is the retrieved context followed by `QUESTION:`, the question and `RESPONSE:`.

> You are the user's personal assistant. You have a memory: excerpts from your earlier conversations with this user, retrieved for the current question. The memory is a record of what was said; the current question decides what to do with it.
>
> **Sources**
>
> – Everything you state about the user - their projects, decisions, plans, preferences, dates, numbers, and what they told you - must come from the memory. Do not invent, embellish, or infer details the memory does not support.
>
> – If the question is about the user's own history or work and the memory does not cover it, say so plainly and name what is missing, instead of guessing or answering with related material. Do not fill such a gap with general knowledge.
>
> – If the question is a general request - an explanation, a calculation, a recommendation, a how-to - you may use your general knowledge to carry it out, shaped by what the memory says about this user.
>
> **Reading the record**
>
> – The memory has an order: messages carry serial numbers, and some carry a date and a session/turn index; larger numbers were said later. Use this order for anything about sequence, timing, or what is current.
>
> – When a later statement updates or replaces an earlier one, the latest is current; bring up the earlier one only when the question is about the history or the change.
>
> – When two statements cannot both be true and neither is framed as a change, do not silently pick one: say what each of them says and ask the user which is right.
>
> – Combine excerpts from different parts of the record when the question needs it. When the answer involves dates, durations, counts, or order, state the specific dates or items you used and how you got the result.
>
> **Answering the user**
>
> – Before answering a request, check the memory for how this user has asked to be answered - format, level of detail, examples, things to include or leave out - and for preferences or constraints that bear on the request. Honor them even when the current question does not repeat them.
>
> – Lead with the answer, and match its length to the question: a short factual question gets a direct one- or two-sentence answer; a request for a summary, explanation, plan, or comparison gets a complete, organized answer that covers each point with the specific details (names, versions, numbers, dates) from the memory.
>
> – Speak to the user directly and naturally. Refer to the memory as your earlier conversations with them, not as excerpts or numbered items.

By category. In Table 14 MUSTER reads U units of the conversation and True Memory its ranked results cut to 9.7K and 19.5K, each the mean of two reads. Mem0 reads its search at 8.2K, the mean of two searches, and HippoRAG-v2 its ranked passages filled to 9.7K and 19.5K. Table 15 gives four rows of Table 6 by category. There Mem0's search at 19.5K and MUSTER on Mem0's memories are

one read each, MUSTER on the conversation reads $U{=}40$ units, and both stores, $U{=}30$ conversation units followed by the memories MUSTER reads from Mem0's store, are the mean of two reads.

Table 14: **BEAM categories at 1M and 10M.** BEAM [21], read by GLM-5.3-Flash [38] under our answer prompt and judged by GPT-4.1-mini with BEAM's code, one store per row. The ten categories are abstention, contradiction resolution, event ordering, information extraction, instruction following, knowledge update, multi-session reasoning, preference following, summarization and temporal reasoning. Bold: MUSTER.

BEAM system	Abst.	Contra.	Order	Extract	Instr.	Update	Multi-s.	Pref.	Summ.	Temp.	All
1M tokens											
Mem0 2.1.0	91.1	69.3	28.0	74.6	94.2	76.1	77.0	90.7	69.1	52.5	72.3
Honcho	97.1	69.8	22.8	55.5	84.4	67.1	54.3	84.5	34.7	36.2	60.7
HippoRAG-v2, 9.7K	90.7	68.2	27.3	81.4	90.2	65.7	69.1	91.3	58.4	43.6	68.6
HippoRAG-v2, 19.5K	91.4	71.8	28.6	83.9	91.4	72.1	74.6	92.7	64.3	42.1	71.3
True Memory, 9.7K	93.9	79.5	27.0	82.0	88.9	76.4	73.5	91.0	57.4	51.5	72.1
True Memory, 19.5K	90.4	78.8	29.4	87.5	90.2	85.4	74.2	93.0	62.0	55.2	74.6
MUSTER, $U{=}20$	**88.9**	**76.8**	**28.4**	**80.8**	**90.2**	**75.0**	**75.4**	**91.1**	**64.9**	**53.1**	**72.4**
MUSTER, $U{=}40$	**83.2**	**75.6**	**28.4**	**85.4**	**91.3**	**77.1**	**76.0**	**94.2**	**68.8**	**53.6**	**73.4**
10M tokens											
Mem0 2.1.0	77.5	65.0	26.1	80.0	92.5	76.2	33.1	74.4	65.9	32.5	62.3
Honcho	90.0	71.2	18.0	57.5	82.5	62.5	14.4	57.1	18.2	17.5	48.9
HippoRAG-v2, 9.7K	90.0	72.5	21.9	65.0	72.5	82.5	27.8	75.0	56.3	23.8	58.7
HippoRAG-v2, 19.5K	85.0	68.8	27.1	63.7	80.0	80.0	39.6	75.0	60.3	20.0	60.0
True Memory, 9.7K	92.5	77.5	24.4	87.5	67.5	92.5	28.9	83.8	45.5	13.8	61.4
True Memory, 19.5K	85.0	77.5	25.8	83.8	86.9	81.3	30.3	82.5	52.8	18.8	62.4
MUSTER, $U{=}20$	**87.5**	**77.5**	**26.0**	**90.0**	**87.5**	**82.5**	**22.3**	**74.6**	**62.9**	**16.3**	**62.7**
MUSTER, $U{=}40$	**85.0**	**78.8**	**28.3**	**87.5**	**83.8**	**80.0**	**43.2**	**71.3**	**64.1**	**21.3**	**64.3**

Build cost at every scale. Table 16 gives the build columns of Table 5 at all three scales. MUSTER builds conversation 0 in under a second at 100K, 7 s at 1M and 77 s at 10M (11.0M source tokens), about 7 s per million source tokens at each scale. Each build time is the mean of three runs with bge-m3 in fp16, without the model load and warm-up. At 10M, MUSTER is 30 times faster than True Memory, the other memory system with no generative call, and the model-written memories take hours. About three quarters of MUSTER's time at each scale (75, 74 and 74%) is the embedding pass, bge-m3 on one RTX PRO 6000 reading at most 2048 tokens of each unit (Appendix A). At 10M MUSTER's five steps take 6 s for the statistics, 9 s for the units, 57 s for the embedding pass, 2 s for the parent graph's entity index and 3 s for BM25.

LM-written counts the tokens a language model wrote into a store, in cl100k tokens (tiktoken's `cl100k_base`), and never the source text. Wall-clock is per conversation unless marked whole scale, at the concurrency we used. Mem0's calls and tokens come from its server's usage meter and log (one GPT-4o-mini call per ingestion chunk). Its store and the source are counted in cl100k tokens from its vector store and the benchmark files. For Honcho, whose model input and store are not observable, we report the tokens we sent and calls estimated from its open-source deriver rule. The rule counts one call per user-message batch of at most 1,024 tokens plus summaries every 20 and 60 messages, applied to the messages we sent. The lower value uses the 4,096-token batch of commit `a1d689b` [31]. Hindsight builds every conversation of a scale in one run with GPT-OSS-120B, served by DeepInfra through OpenRouter at 100K and with vLLM on one RTX PRO 6000 at 1M and 10M. Its calls and input tokens are counted per call at its model server, and its facts and entity names in its database, without the copy of the names it keeps beside each fact for BM25. At 10M its fact tokens and entity names come from a database snapshot that holds the final stores of nine conversations, at 31.0 tokens per fact, and the tenth is scaled by its fact count. EverMemOS builds every conversation of a scale in one run with GPT-OSS-120B on one RTX PRO 6000 with vLLM, and its LM-written count takes each MemCell's episode, summary, subject and time and its atomic facts. At 100K its calls, input and wall-clock cover the first pass over all 20 conversations and the re-run of one of them. At 1M, calls and input come from the first pass over all 35 conversations, and the wall-clock includes one conversation re-run and the indexing. At 10M, calls, input, wall-clock and tokens stored cover the whole build of the ten conversations. HippoRAG-v2 builds every conversation of a scale in one run on one RTX PRO 6000, with OpenIE by GPT-OSS-120B on vLLM and then indexing with NV-Embed-v2. Its OpenIE makes an entity call and a triple call per pair of turns, counted at a metering proxy. Its entity nodes and relations are rebuilt from its saved OpenIE output with its own index code, which reproduces the 624,468 entity nodes and 1,360,943

relations of its ten 10M graphs. True Memory builds conversation 0 with its released BEAM worker, which adds one message at a time, on one RTX PRO 6000.

Build cost on MemoryAgentBench. MUSTER builds memory from all 24.4M source tokens of the benchmark's fourteen stores in 150.1 s, 6.1 s per million source tokens counted in cl100k, against about 7 s on BEAM. The build time is the mean of three runs without the model load and warm-up. The stores hold 126 source texts of 194K tokens on average, and MUSTER builds each into its own memory in 1.2 s on average and 0.8 s at the median. The same five steps run as on BEAM, with the embedding pass in fp16 on one RTX PRO 6000. InfBench-Sum's novels hold 72% of the source tokens, and their build time grows in proportion to length ($R^2 = 0.998$). The other thirteen stores take 35.7 s together, each under 9 s.

Table 17 gives the build cost of the model-written memories we run in Table 1 on the same 126 source texts, with the memory-writing models of Table 31. Hindsight and EverMemOS call GPT-OSS-120B served with vLLM on one RTX PRO 6000 and build all 126 memories in one run, with calls and input counted at a metering proxy. Mem0's calls and tokens come from its server's usage meter, one call per chunk (5,956 with either writer). With GPT-4o-mini it runs in eight consecutive parts, each building 1 to 20 source texts at once, and its wall-clock is the sum of the parts. With GPT-OSS-120B, served by DeepInfra through OpenRouter, it runs 24 source texts at once. Honcho is the hosted service, run on one source text and then on 12 at a time, and timed until its ingestion queue drains. MUSTER builds one memory at a time.

Table 15: **Both stores by BEAM category.** Four rows of Table 6 at 1M and 10M, read by GLM-5.3-Flash under our answer prompt and judged by GPT-4.1-mini with BEAM's code. The first two columns name what searches the conversation and Mem0's memories (–, not searched). Δ: MUSTER on both stores minus Mem0's search. Categories as in Table 14. Bold: MUSTER.

Over the conversation	Over Mem0's memories	Abst.	Contra.	Order	Extract	Instr.	Update	Multi-s.	Pref.	Summ.	Temp.	All
1M tokens												
–	Mem0's search	90.0	68.9	28.8	79.4	93.6	80.0	75.4	92.1	71.5	50.7	73.1
–	**MUSTER**	**91.4**	**73.2**	**28.3**	**81.4**	**92.6**	**79.6**	**73.7**	**94.9**	**70.4**	**50.4**	**73.6**
MUSTER, $U{=}40$	–	**83.2**	**75.6**	**28.4**	**85.4**	**91.3**	**77.1**	**76.0**	**94.2**	**68.8**	**53.6**	**73.4**
MUSTER, $U{=}30$	**MUSTER**	**86.1**	**72.9**	**30.0**	**90.9**	**95.5**	**84.3**	**79.3**	**94.1**	**70.7**	**51.8**	**75.6**
Δ		−3.9	+4.0	+1.2	+11.5	+1.9	+4.3	+3.9	+2.0	−0.8	+1.1	+2.5
10M tokens												
–	Mem0's search	75.0	71.2	30.8	82.5	91.2	72.5	26.8	87.5	64.3	37.5	63.9
–	**MUSTER**	**80.0**	**68.8**	**31.4**	**78.8**	**91.2**	**85.0**	**52.3**	**80.0**	**68.8**	**27.5**	**66.4**
MUSTER, $U{=}40$	–	**85.0**	**78.8**	**28.3**	**87.5**	**83.8**	**80.0**	**43.2**	**71.3**	**64.1**	**21.3**	**64.3**
MUSTER, $U{=}30$	**MUSTER**	**82.5**	**76.6**	**28.8**	**90.0**	**81.9**	**86.2**	**32.5**	**75.4**	**69.3**	**38.8**	**66.2**
Δ		+7.5	+5.4	−2.0	+7.5	−9.3	+13.7	+5.7	−12.1	+5.0	+1.3	+2.3

Qwen3.5-9B at 9.7K tokens. Under BEAM's answer prompt, MUSTER at 20 units minus each memory system on the three-scale mean of Table 4, with whole conversations resampled 10,000 times, is +2.92 ([+0.13, +5.87]) against Mem0, +10.17 ([+7.84, +12.57]) against Honcho, +5.80 ([+3.11, +8.71]) against Hindsight and +2.39 ([+0.15, +4.86]) against EverMemOS.

Mem0 at matched budgets. Re-running Mem0's own search with a larger limit and cutting its list at the reader budgets of 20, 30 and 40 units (9.7K, 14.6K, 19.5K tokens) gives 76.2, 76.0 and 75.5 at 100K, and its re-searched top-200 gives 75.6 (the saved search gave 75.4). Mem0 does not gain from more context at 100K, where its top-200 is already 58% of its store. At 10M the same budgets give Mem0 62.2, 62.9 and 63.9. MUSTER gives 75.0 and 76.1 at 20 and 40 units at 100K and 62.7 and 64.3 at 10M. Mem0's search is not deterministic (a second search overlaps the first top-200 by 0.96, 0.86 and 0.61 at the three scales and scores 75.6, 72.7 and 61.5 against 75.4, 71.8 and 63.2). Table 5 reports the mean of the two.

The rule's other branch. With recshare below 0.90, the rule would read BEAM as it reads the novels of MemoryAgentBench, as sentence-bounded 1024-token units under 8192-token parents (Section 3.5). On BEAM-100K, ten such units (10.1K tokens) score 73.7, and 20 sentence-bounded 512-token units that ignore record boundaries, under the same 4096-token parents as MUSTER's,

Table 16: **MUSTER builds memory in seconds.** Building memory for one BEAM [21] conversation at each scale. Below each system, the model that writes its memory (Table 31). LM input: tokens read by the memory-writing model. LM-written: tokens it wrote into the store. ‖Conversation 0 on one RTX PRO 6000, no generative call, and for MUSTER the mean of three runs. ¶Per-call input at 1M times the 10M calls. §Estimated from Honcho's open-source rules. *All conversations of a scale in one run. For EverMemOS, the mean uses recorded conversation completions on the active build clock. †Served by DeepInfra through OpenRouter. ‡Counted in nine of the ten conversations, the tenth scaled by its fact count. Bold: MUSTER.

		100K	1M	10M
	source tokens	132K	1.05M	11.0M
Mem0 2.1.0	LM calls	143	1,066	10,429
GPT-4o-mini	LM input (× source)	1.38M (10.4×)	10.3M (9.8×)	≈101M (9.2×)¶
	memories	456	3,432	32,490
	LM-written tokens (% source)	14.9K (11.3%)	102K (9.7%)	1.01M (9.2%)
	top-200 tokens (% of store)	8.7K (58%)	8.2K (8.0%)	8.5K (0.8%)
	wall-clock	11 min	77 min	13.0 h
Honcho	tokens sent	130K	1.03M	10.8M
service models	model calls (estimated§)	54–155	428–1,190	4,441–11,700
	wall-clock (with queue drain)	7 min	28 min	7.8 h
Hindsight	LM calls	222	1,645	16,425
GPT-OSS-120B	LM input (× source)	0.87M (6.6×)	6.53M (6.2×)	65.5M (6.0×)
	facts / entity names	1,554 / 445	10,431 / 1,594	108,147 / ≈7.8K‡
	LM-written tokens (% source)	53.2K (40.3%)	326K (31.2%)	≈3.38M (≈30.7%)‡
	wall-clock, whole scale*	65 min†	4.1 h	12.3 h
	per conversation (mean)	49 min†	30 min	3.0 h
EverMemOS	LM calls	364	2,713	26,080
GPT-OSS-120B	LM input (× source)	1.32M (10.0×)	9.87M (9.4×)	106M (9.7×)
	MemCells	53	444	3,765
	LM-written tokens (% source)	49.2K (37.3%)	440K (42.0%)	3.85M (35.0%)
	wall-clock, whole scale*	60 min	12.0 h	39.8 h
	per conversation (mean), all at once	43 min	7.7 h	32.7 h
HippoRAG-v2	LM calls	287	2,132	20,870
GPT-OSS-120B	LM input (× source)	0.38M (2.8×)	2.94M (2.8×)	30.1M (2.7×)
	entity nodes / relations	1,351 / 2,005	9,068 / 16,138	62,447 / 136,094
	LM-written tokens (% source)	20.3K (15.4%)	163K (15.5%)	1.35M (12.2%)
	wall-clock, whole scale*	20 min	3.5 h	13.0 h
	per conversation (mean)	1.0 min	6.0 min	1.3 h
True Memory	generative calls / input tokens	0 / 0	0 / 0	0 / 0
none	build time, one GPU‖	8.2 s	200 s	2,325 s
MUSTER	**generative calls / input tokens**	**0 / 0**	**0 / 0**	**0 / 0**
none	**build time, one GPU‖**	**0.9 s**	**6.8 s**	**76.8 s**

Table 17: **MUSTER builds memory for MemoryAgentBench in** 150 **seconds.** All 126 source texts (24.4M source tokens). LM input: tokens read by the memory-writing model, and for Honcho the tokens we sent. One GPU is one RTX PRO 6000. Bold: MUSTER. n/o, not observable.

System	memory-writing model	LM calls	LM input (× source)	runs on	wall-clock	s per 1M source tokens
Mem0 2.1.0	GPT-4o-mini	5,956	75.0M (3.1×)	API	3.4 h	508
Mem0 2.1.0	GPT-OSS-120B	5,956	76.6M (3.1×)	API	4.1 h	607
Honcho	service models	n/o	24.5M sent	service	3.7 h	543
Hindsight 0.10.3	GPT-OSS-120B	50,995	176M (7.2×)	one GPU	3.2 h	478
EverMemOS	GPT-OSS-120B	11,936	64.2M (2.6×)	one GPU	3.4 h	495
MUSTER	**none**	**0**	**0**	**one GPU**	**150.1 s**	**6.1**

score 74.4 (9.9K). Both reads of the record units MUSTER detects, 75.3 and 74.6 at 9.7K tokens, are above both, with less context. Two unrefined 4096-token chunks score 57.7 (Table 23), so the large loss comes from the coarse unit, not from cutting across records.

MUSTER with a model-built store. We number Mem0's memories one per line in write order, so serial reads 1.0 and the rule takes the serial branch, with one memory per unit, parent-priority floors $(2, 0, 8)$, 200 memories after the source units and 400 when read alone, each shown as `[serial] [date] text`. MUSTER's assembly (last row of Table 6) shows its 30 source units, then that selection, admitted in reranker order until the per-question budget is reached and shown in write order, so each source keeps its own search. With Mem0's own search in the same place the assembly scores 77.3, 73.3 and 64.2 against 77.7, 75.6 and 66.2 (difference $+1.6$), so the gain comes from which memories are chosen. On the three-scale mean the gain is $+2.3$ over Mem0's search (paired 95% interval $[+0.3, +4.5]$) and $+1.9$ over MUSTER at 40 units ($[+0.4, +3.3]$), each draw resampling the conversations of every scale. Of the other assemblies we compared at the same budget, letting each memory bring the source chunk it was written from, before or beside it, ties MUSTER's assembly on the mean (73.1), and the others score 71.0 to 72.9. The two searches point at the same parents for 94% of the top ten memories at 100K and 61% at 10M, so at scale the memory store reaches evidence the source search does not. When the source leads, every memory summarises a chunk already shown, and that assembly is the lowest. Merging the two into one store (the conversation followed by every Mem0 memory) and letting MUSTER select from it scores 77.0 at 40 units but 73.0 at 20 units at 100K. At a small budget the short memories take slots from the records, so a fixed share for each source works better than one pool.

Hindsight at three scales. Hindsight 0.10.3 writes each session with GPT-OSS-120B, as in the OSS-120B configuration of its paper, served by DeepInfra (bf16) through OpenRouter, and ingests BEAM-100K the way its benchmark harness does, with one bank per conversation, one document per session with BEAM's time anchors in the text, the harness's retain instructions and no observations. It makes 4,438 model calls for the twenty conversations. We read its default recall (budget *mid*, 4,096 tokens, 7.3K tokens after rendering), and it scores 66.4 with GLM-5.3-Flash and 60.2 with Qwen3.5-9B under our answer prompt, and 40.4 with Qwen3.5-9B under BEAM's, all with BEAM's judge. At 1M the same write path runs on one RTX PRO 6000 with GPT-OSS-120B served by vLLM and makes 57,588 model calls for the 35 conversations in 4.1 hours. Hindsight's default recall gives 8.4K tokens, and it scores 67.2 with GLM-5.3-Flash and 59.9 with Qwen3.5-9B under our answer prompt, and 40.5 under BEAM's. At 10M it makes 164,249 model calls for the ten conversations in 12.3 hours. Its default recall gives 9.2K tokens, and it scores 52.2 with GLM-5.3-Flash and 48.2 with Qwen3.5-9B under our answer prompt, and 34.2 under BEAM's.

EverMemOS at 100K, 1M and 10M. EverMemOS runs its released code at `fe436d3` with GPT-OSS-120B for memory construction and agentic retrieval at every scale, served by vLLM on one RTX PRO 6000 beside Qwen3-Embedding-4B and Qwen3-Reranker-4B, over BEAM's conversations with their session dates. The writer thus differs from the GPT-4.1-mini judge. At 100K construction makes 7,276 model calls over the twenty conversations and retrieval 612. Six answers needed the parse-only repair. We read the context its answer stage assembles (4.8K tokens), and it scores 72.0 with GLM-5.3-Flash and 61.9 with Qwen3.5-9B under our answer prompt, and 44.2 with Qwen3.5-9B under BEAM's, all with BEAM's judge. Built and searched with GPT-4.1-mini, the model of its paper, it scores 72.0, 63.2 and 45.8. At 1M it forms 15,527 MemCells over the 35 conversations. The first pass takes 10.2 hours and finishes 34 of them, since EverMemOS drops a conversation when one MemCell's event log comes back empty five times. The last one, rebuilt alone with the same settings, takes 1.8 hours. EverMemOS needed the parse-only repair on 33 answers. Its answer stage assembles 4.5K tokens, and it scores 69.4 with GLM-5.3-Flash and 62.6 and 43.6 with Qwen3.5-9B under our and BEAM's answer prompts. At 10M the same release, model and server form 37,650 MemCells over the ten conversations, built at once, in 39.8 hours of construction, each finishing after 27.3 to 38.2 hours, and 330 answers needed the parse-only repair. Its answer stage assembles 4.8K tokens, and it scores 60.6 with GLM-5.3-Flash and 55.1 and 37.5 with Qwen3.5-9B under our and BEAM's answer prompts. Given more context, with its retrieval returning up to 20 memories, its maximum (7.3K tokens at 100K), it scores 73.1, 70.7 and 59.0 with GLM-5.3-Flash under our prompt (mean 67.6, against 67.3 with ten). Cut from its own 100 reranked candidates to MUSTER's two contexts, it scores 70.1 and 70.7 on the mean (Table 22).

HippoRAG-v2 at three scales. HippoRAG-v2 builds one graph per conversation with GPT-OSS-120B (Table 16), where its paper uses Llama-3.3-70B-Instruct, and is read from its own ranking of the conversation's passages, whole passages in rank order filled to 9.7K and 19.5K tokens, so the first five passages of every context are the five it returns by default. With GLM-5.3-Flash under our answer prompt it scores 72.1, 68.6 and 58.7 at 9.7K and 75.7, 71.3 and 60.0 at 19.5K, and with Qwen3.5-9B at 9.7K 64.0, 62.2 and 49.6 under our answer prompt and 46.9, 43.8 and 38.3 under BEAM's, all with BEAM's judge.

Write model. Table 31 lists the model that writes each system's memory on each benchmark. Writing Mem0's and EverMemOS's memories with GPT-OSS-120B (reasoning effort low) in place of the models of their papers lowers Mem0's GLM-5.3-Flash score at 100K by 0.2 and leaves EverMemOS's unchanged. Mem0 2.1.0 ingesting with GPT-OSS-120B (served by DeepInfra through OpenRouter) stores 7,330 memories over the twenty conversations where GPT-4o-mini stores 9,116, and scores 75.2 from its saved top-200 search (75.4 with GPT-4o-mini), and 63.9 with Qwen3.5-9B under our answer prompt at 9.7K (62.9 with GPT-4o-mini). EverMemOS with GPT-OSS-120B for construction and retrieval (served by vLLM on one RTX PRO 6000, with its three retrieval calls, capped at 500, 150 and 300 tokens, given 1,024 more for the reasoning) forms 1,053 MemCells where GPT-4.1-mini forms 1,441. The GPT-OSS-120B build scores 72.0 with GLM-5.3-Flash, as with GPT-4.1-mini, and 61.9 and 44.2 with Qwen3.5-9B under our and BEAM's answer prompts (63.2 and 45.8 with GPT-4.1-mini). All scores use BEAM's judge. On MemoryAgentBench, Mem0 2.1.0 written with GPT-OSS-120B (DeepInfra through OpenRouter, reasoning effort low, its 2,000-token output allowance raised by 1,024 for the reasoning) scores 28.6 against 26.7 with GPT-4o-mini (Table 1). Of the GPT-OSS-120B run's 5,956 extraction calls, 191 still end at that limit, and 369 chunks store no memory against 142.

Mem0's published answers under our judge. Mem0's published BEAM answers from its managed platform [53] (GPT-5 answerer and top-200, reported by the vendor at 64.1 and 48.6 under its own GPT-5 judge) score 63.1 at 1M and 54.8 at 10M under BEAM's official judge with GPT-4.1-mini. Our open-source run scores 72.3 and 62.3 (mean of two searches).

D THE MEMORY SYSTEMS AT THE READERS OF THE PER-BENCHMARK TABLES

Table 7 reads GLM-5.3-Flash at high reasoning effort on MemoryAgentBench and at maximum elsewhere, and GPT-6 Luna at high, both under the answer prompt and judge of each benchmark in Table 20. Table 18 gives the paired tests behind Section 5.5. On MAB the difference is the mean of the four competencies, tested by flipping signs within each cell's questions (20,000 flips), on LoCoMo-Refined by the exact McNemar test, and on BEAM-10M the difference is the category-balanced mean, tested by flipping signs over questions.

Context in Table 7. Table 19 gives each system's context per question. Each system reads the contexts of Section 5.1 and Tables 3 and 5. These are cut to MUSTER's per-question budget where the system takes one and follow the system's own default otherwise. The default applies to Mem0 on MemoryAgentBench (its top 100 memories, as the benchmark reads it), to EverMemOS and Hindsight on LoCoMo-Refined, and to Hindsight and Honcho on BEAM-10M. On MemoryAgentBench the rule is Table 1's, HippoRAG-v2 included. Each row is cut at MUSTER's budget, never below one whole unit, and on FactConsolidation each system reads its own uncut retrieval. On MemoryAgentBench as a whole, True Memory and HippoRAG-v2, which read as much as MUSTER or more, score 7.0 to 9.4 points lower. Mem0's whole store is smaller than MUSTER's budget on the three novel cells (7.5K to 18.9K against 40.3K to 40.5K), and filled to that budget it scores 26.4 against 26.7 at its top 100 with GPT-4o-mini (Table 1). Reader cost per 1,000 questions at list price is $2.80 for MUSTER and $94.91 for the full store.

Table 20 gives the same systems at the readers of Tables 1, 3 and 5. GPT-4o-mini reads MemoryAgentBench, where All averages the four competencies. GPT-4.1-mini reads LoCoMo under Mem0's and EverMemOS's answer prompts at MUSTER's ten units, with Mem0, True Memory and HippoRAG-v2 held to them, Honcho given them through its context API and the others at their own. GPT-4o-mini reads LoCoMo-Refined under its official evaluation, every store rebuilt from its files

Table 18: **Paired tests behind Table 7.** MUSTER minus each row, two-sided p in parentheses. *As in Table 7.

	MAB		LoCoMo-Refined		BEAM-10M	
Row	GLM	Luna	GLM	Luna	GLM	Luna
Full store*	+3.2 (0.005)	+3.2 (0.007)	−0.1 (0.93)	+0.4 (0.65)	+30.0 (<0.001)	+28.4 (<0.001)
True Memory	+9.4 (<0.001)	+8.4 (<0.001)	+8.7 (<0.001)	+9.4 (<0.001)	+1.9 (0.36)	+2.6 (0.30)
Mem0	+33.9 (<0.001)	+33.6 (<0.001)	+8.2 (<0.001)	+9.3 (<0.001)	+0.4 (0.89)	−1.3 (0.65)
EverMemOS	+32.1 (<0.001)	+31.1 (<0.001)	+8.5 (<0.001)	+8.2 (<0.001)	+1.6 (0.50)	+1.1 (0.65)
HippoRAG-v2	+9.1 (<0.001)	+7.0 (<0.001)	+12.7 (<0.001)	+15.0 (<0.001)	+4.4 (0.073)	+5.1 (0.052)
Hindsight	+29.1 (<0.001)	+26.8 (<0.001)	+17.9 (<0.001)	+18.7 (<0.001)	+12.1 (<0.001)	+9.3 (0.002)
Honcho	+26.0 (<0.001)	+25.7 (<0.001)	+23.2 (<0.001)	+25.7 (<0.001)	+15.4 (<0.001)	+10.9 (<0.001)

Table 19: **Context behind Table 7.** Tokens per question in each reader's own tokenizer, mean of the two readers (Tables 3 and 5 count cl100k). °The system's own default. Every other memory-system cell is held to MUSTER's budget (4,900 tokens on LoCoMo-Refined, about MUSTER's ten units). *As in Table 7. Bold: MUSTER.

System	MAB	LoCoMo-Refined	BEAM-10M
MUSTER	**10K**	**5.2K**	**20K**
Full store*	310K	31K	849K
True Memory	14K	5.1K	20K
Mem0	3.0K°	5.2K	20K
EverMemOS	3.3K	4.3K°	19K
HippoRAG-v2	10K	5.1K	20K
Hindsight	6.8K	4.9K°	10K°
Honcho	7.3K	5.5K	15K°

and the memory systems written with GPT-OSS-120B, Honcho with its service's models. GLM-5.3-Flash reads BEAM-10M at 19.5K with our answer prompt, Honcho and Hindsight at their own context, and Tables 4 and 5 give all three scales and both prompts (MUSTER's three-scale mean is 71.3). The last column counts the language-model calls that build a 10M-token memory.

Table 20: **MUSTER leads at each benchmark's reader.** The systems on MemoryAgentBench (MAB) [1], LoCoMo [35], LoCoMo-Refined [41] and BEAM [21] at the reader, judge and answer prompt of each per-benchmark table, named above the scores (our prompt in Appendix C). Cell metrics: each MAB cell's own metric (Section 4). Bold: MUSTER. †Built with GPT-OSS-120B in place of the model of the system's paper (Table 31). ‡Reported by MAB. –, not run. AML, Agent Memory Leaderboard.

	MAB, All	LoCoMo		LoCoMo-Refined	BEAM-10M	LM calls,
Reader	GPT-4o-mini	GPT-4.1-mini		GPT-4o-mini	GLM-5.3-Flash	10M build
Judge	cell metrics	GPT-4o-mini		Qwen3-14B	GPT-4.1-mini	
Answer prompt	MAB's	Mem0's	EverMemOS's	AML's	ours	
Mem0 2.1.0 [32]	26.7	80.50	89.44	73.73†	63.9	10,429
Honcho [31]	35.0	67.34	77.62	56.44	48.9	4.4–11.7K
Hindsight 0.10.3 [33]	31.4	73.48	85.35	63.68	52.2	16,425
EverMemOS [36]	23.1†	83.87	93.05	75.33†	62.7†	26.1K
HippoRAG-v2 [14]	41.6‡	75.09†	84.46†	67.58†	60.0†	20,870
True Memory [30]	43.6	81.93	90.58	71.56	62.4	0
MUSTER	**57.5**	**86.04**	**93.46**	**77.57**	**64.3**	**0**
MUSTER, beside Mem0's memories	–	–	–	–	**66.2**	**10,429**

E OTHER READERS AND BEAM'S OWN ANSWER PROMPT

Table 21 reruns the Table 5 instrument under BEAM's answer prompt (Listing 44 of [21], a direct answer without explanation) and with the open-weight Qwen3.5-9B, on the same retrieved contexts. Our answer prompt (Appendix C) raises every system. At U=40 MUSTER is above or level with Mem0's default search at all three scales under every reader and prompt, and above Honcho everywhere. Qwen3.5-9B runs in bf16 with thinking off. MUSTER uses its default renderer, Mem0 its own default search and True Memory its ranked results cut to 9.7K. Under GLM-5.3-Flash with our prompt the Mem0 cells average two searches and the True Memory cells two reads, and under Qwen3.5-9B with our prompt MUSTER's 10M cells average two reads. Table 22 gives MUSTER,

Mem0, True Memory, EverMemOS and HippoRAG-v2 per scale at the same reader budget. There, under GLM-5.3-Flash with our prompt the MUSTER and True Memory cells, and under Qwen3.5-9B with our prompt the 10M cells of MUSTER and Mem0, are the mean of two reads. True Memory's ranked results and EverMemOS's own 100 candidates, reranked by its reranker, are cut to the budget, and HippoRAG-v2's ranked passages are filled to it. Under the judge MemoryLACE uses (GPT-4o-mini), Qwen3.5-9B with our answer prompt scores MUSTER 58.8 at $U{=}20$ and 61.1 at $U{=}40$ and Mem0 56.7 at BEAM-100K, and Hindsight written with GPT-OSS-120B 52.1. MemoryLACE reports 51.9 for its own system, with its own prompt and FP8 weights [22].

Table 21: **Our answer prompt raises every system.** BEAM [21] with GLM-5.3-Flash [38] and Qwen3.5-9B [39], each under our answer prompt (Appendix C) and BEAM's, judged by GPT-4.1-mini with BEAM's code. True Memory reads 9.7K and Mem0 its default search. Each cell lists 100K / 1M / 10M. Bold: MUSTER.

Reader, answer prompt	Mem0	Honcho	True Memory	**MUSTER, $U{=}20$**	**MUSTER, $U{=}40$**
GLM-5.3-Flash, ours	75.5 / 72.3 / 62.3	67.1 / 60.7 / 48.9	75.4 / 72.1 / 61.4	**75.0 / 72.4 / 62.7**	**76.1 / 73.4 / 64.3**
GLM-5.3-Flash, BEAM's	60.4 / 58.0 / 49.6	56.1 / 49.9 / 40.8	59.4 / 58.6 / 48.3	**57.8 / 57.4 / 48.4**	**61.0 / 58.7 / 49.8**
Qwen3.5-9B, ours	64.2 / 63.2 / 51.8	58.5 / 53.8 / 42.6	66.4 / 63.0 / 50.2	**65.3 / 62.5 / 53.1**	**66.2 / 64.3 / 52.0**
Qwen3.5-9B, BEAM's	42.3 / 41.0 / 37.2	40.2 / 34.2 / 27.6	47.4 / 44.1 / 36.3	**48.7 / 43.9 / 39.9**	**47.4 / 46.1 / 37.2**

Unrefined 4096-token chunks. The last row of Table 23 cuts BEAM-100K into sentence-bounded 4096-token chunks, the reference scale without any refinement, and keeps two of them under the balanced floors $(4, 3, 3)$, which at two slots leave the reranker's top two. Two chunks (8.1K tokens) score 57.7, 17.3 below 20 record units at 9.7K (95% CI $[+13.3, +21.1]$, 19 of 20 conversations). The loss concentrates in information extraction (-41.7), knowledge update (-36.2) and temporal reasoning (-30.3).

Table 22: **Equal budgets at each BEAM scale.** BEAM, Table 4 per scale, read under the answer prompts of Table 21 and judged by GPT-4.1-mini with BEAM's code. Each cell lists 100K / 1M / 10M. Bold: MUSTER. –, not run.

		GLM-5.3-Flash	Qwen3.5-9B	
budget	store read, retrieval	our prompt	our prompt	BEAM's prompt
9.7K	Mem0 2.1.0, full-ver.	76.2 / 72.9 / 62.2	62.9 / 62.9 / 50.4	44.7 / 43.1 / 35.9
9.7K	EverMemOS	75.0 / 71.7 / 63.7	–	–
9.7K	HippoRAG-v2	72.1 / 68.6 / 58.7	64.0 / 62.2 / 49.6	46.9 / 43.8 / 38.3
9.7K	True Memory	75.4 / 72.1 / 61.4	66.4 / 63.0 / 50.2	47.4 / 44.1 / 36.3
9.7K	**MUSTER, $U{=}20$**	**75.0 / 72.4 / 62.7**	**65.3 / 62.5 / 53.1**	**48.7 / 43.9 / 39.9**
19.5K	Mem0 2.1.0, full-ver.	75.5 / 73.1 / 63.9	63.2 / 64.0 / 54.3	43.9 / 43.1 / 36.1
19.5K	EverMemOS	76.3 / 73.1 / 62.7	–	–
19.5K	HippoRAG-v2	75.7 / 71.3 / 60.0	–	–
19.5K	True Memory	78.1 / 74.6 / 62.4	68.9 / 65.8 / 57.6	47.3 / 44.1 / 35.8
19.5K	**MUSTER, $U{=}40$**	**76.1 / 73.4 / 64.3**	**66.2 / 64.3 / 52.0**	**47.4 / 46.1 / 37.2**

Table 23: **MUSTER's detected unit scores highest.** BEAM-100K [21] by unit at about 10K tokens of context, read by GLM-5.3-Flash under our answer prompt and judged by GPT-4.1-mini with BEAM's code. Bold: MUSTER, the mean of two reads.

unit	context	BEAM-100K
MUSTER, record units, 512 tokens	**9.7K**	**75.0**
the rule's cut for stores without records, 1024-token units, 8192-token parents	10.1K	73.7
sentence cut, 512 tokens, record boundaries ignored	9.9K	74.4
unrefined 4096-token chunks	8.1K	57.7

F LoCoMo: PROTOCOL AND FURTHER RESULTS

Data and store. We use LoCoMo's `locomo10.json` and its four answerable categories, numbered as LoCoMo numbers them, with 282 multi-hop, 321 temporal, 96 open-domain and 841 single-hop questions. The fifth category has no gold answer. Each turn is written as one blank-line record holding its dialogue id, its session date, the speaker, the text and, where the turn shares

an image, the image caption, and nothing else is added. As for BEAM, $c = 4096$ because LoCoMo prescribes no chunk. Two other ways of writing the same conversations, LoCoMo's own (a date header per session, then *X said*) and one speaker-prefixed line per turn, give the same branch with 1,024-token parents.

Floors below ten units. The balanced floors $(4, 3, 3)$ fill ten slots, so five units need smaller ones. Below ten units we keep the two dense anchors, give each other channel one slot, and share the rest in the ratio 4:3:3 by largest remainder, ties going in the fill order (dense, graph, BM25). Five units get $(3, 1, 1)$. The same rule returns $(4, 3, 3)$ at ten units and $(4, 3, 3) \cdot U/10$ at every multiple of ten, and with it switched on all 1,540 contexts at ten units are byte-identical to those without it.

Prompts and judges. Mem0's answer prompt is the single-list prompt of Mem0's LoCoMo evaluation code (`ANSWER_PROMPT_ZEP`, commit `aae5989`), because a MUSTER block holds both speakers. The judge is `ACCURACY_PROMPT` of the same code. EverMemOS's answer prompt, context template and judge come from its evaluation code at commit `fe436d3`, and its answer is the text after `FINAL ANSWER:`. Readers and the GPT-4o-mini and GPT-4.1-mini judges run at temperature 0. Claude Haiku 5.5 and GPT-5-mini (minimal reasoning) accept no temperature and run at the API's defaults. We report the mean of three runs per judge, one for GPT-5-mini.

Systems compared. EverMemOS runs its released code at `fe436d3` with GPT-4.1-mini for construction, retrieval and answering, as in its paper, and Qwen3-Embedding-4B and Qwen3-Reranker-4B served by vLLM. EverMemOS's three judge runs score 92.99, 93.12 and 93.05, and by category 96.31, 92.79, 89.20 and 78.12 against the 96.67, 91.84, 89.72 and 76.04 it reports. Mem0 2.1.0 runs as a self-hosted server under Mem0's own LoCoMo runner with GPT-4o-mini extraction, as in its paper, and each memory is shown with its session date. We read its top 20 memories (0.7K tokens) and its list cut to 4,900 tokens. Hindsight 0.10.3 retains with GPT-OSS-120B, as in the OSS-120B configuration of its paper, served by DeepInfra through OpenRouter, and is read at its default recall (budget *mid*, 4,096 tokens), rendered as its benchmark harness renders it. Honcho runs as the hosted service under its repository's LoCoMo runner (`tests/bench/locomo.py` at `258fd37`), with one workspace and one session per conversation, both speakers as peers, each message dated by its session, and one dream per speaker once the ingestion queue has drained. Honcho's session-context API is given MUSTER's 4,900 tokens, as on MemoryAgentBench, and returns 5.2K (its three judge runs score 67.27, 67.21 and 67.53, and 77.60, 77.73 and 77.53). True Memory 0.7.6.2 runs its released code at `063e5b8` in its Pro configuration, with its HyDE expansion generated by GPT-OSS-120B on one RTX PRO 6000. True Memory returns 100 results per question (6.2K tokens), which we read whole and cut, in rank order, to 4,900 tokens. Without HyDE it calls no model, and its results, cut the same way, are read under both prompts. HippoRAG-v2 runs its released code at `d5c8329` with NV-Embed-v2 and with GPT-OSS-120B (reasoning effort low, served by vLLM on one RTX PRO 6000) for its entity and triple extraction and for the fact filter it runs per question, which its paper runs with Llama-3.3-70B-Instruct. HippoRAG-v2's output allowances under 4,096 tokens get 1,024 more for the reasoning, and its extraction answers follow the JSON schema of its prompt's example. Each session is one passage, and we read its top three sessions (2.8K tokens) and, in Table 3, its ranked sessions filled to 4,900 tokens. To build and search memory EverMemOS reads 12.9M input tokens, Mem0 50.3M, Hindsight 2.2M, HippoRAG-v2 5.0M and True Memory 0.21M. Honcho's calls are not observable, and its input is at least the 0.18M tokens we send.

Where the lead over the whole conversation comes from. Under Mem0's answer prompt MUSTER at ten units scores 11.3 points above the whole conversation on temporal questions (85.67 against 74.35), 6.6 on open-domain, 2.0 on single-hop and 0.6 on multi-hop. Temporal questions, 321 of the 1,540, give 59% of MUSTER's 3.98-point lead.

Table 24 gives each store with its reader context, the mean number of tokens given to the reader per question. Table 25 rescores the MUSTER and whole-conversation answers under four judges and adds GPT-4o-mini as reader. Under Mem0's answer prompt the score falls as units are added, from 86.04 at ten to 82.97 at forty and 82.06 for the whole conversation. Under EverMemOS's answer prompt it stays between 93.46 and 94.05 from ten units on, above the whole conversation's 93.44. The lead over the whole conversation under Mem0's prompt holds under every judge, by 3.98 with GPT-4o-mini, 4.04 with GPT-4.1-mini, 4.94 with GPT-5-mini and 3.87 with Claude Haiku 5.5. With GPT-4o-mini as reader the lead is 3.96 points under Mem0's prompt and 4.22 under EverMemOS's.

Table 24: **Ten units match or beat the full store.** LoCoMo [35] stores with their reader context, read by GPT-4.1-mini [40] and judged by GPT-4o-mini [45]. Full store: the whole conversation, unretrieved. Bold: MUSTER.

	Reader	Answer prompt	
Store read, retrieval	context	Mem0's	EverMemOS's
Mem0 2.1.0, top 20	0.7K	72.99	84.22
Mem0 2.1.0, cut to 4,900 tokens	4.7K	80.50	89.44
Honcho, context API given 4,900 tokens	5.2K	67.34	77.62
Hindsight 0.10.3, default recall	5.0K	73.48	85.35
EverMemOS	2.4K	83.87	93.05
HippoRAG-v2, top 3 sessions	2.8K	68.14	78.05
HippoRAG-v2, filled to 4,900 tokens	4.9K	75.09	84.46
True Memory 0.7.6.2, all 100 results	6.2K	82.23	91.00
True Memory 0.7.6.2, cut to 4,900 tokens	4.9K	81.93	90.58
True Memory 0.7.6.2, no HyDE, cut to 4,900 tokens	4.9K	81.19	90.35
Full store	31.1K	82.06	93.44
MUSTER, $U{=}5$	**2.5K**	**85.74**	**92.25**
MUSTER, $U{=}10$	**5.0K**	**86.04**	**93.46**
MUSTER, $U{=}20$	**10.0K**	**84.78**	**94.05**
MUSTER, $U{=}40$	**19.8K**	**82.97**	**93.48**

Table 25: **LoCoMo under four judges.** MUSTER ($U{=}10$) and the full store on LoCoMo [35], by reader and judge. Readers are GPT-4.1-mini and GPT-4o-mini, and judges GPT-4o-mini, GPT-4.1-mini, GPT-5-mini and Claude Haiku 5.5. Bold: MUSTER.

		Mem0's prompt		EverMemOS's prompt	
Reader	Judge	**MUSTER**	Full store	**MUSTER**	Full store
GPT-4.1-mini	GPT-4o-mini	**86.04**	82.06	**93.46**	93.44
GPT-4.1-mini	GPT-4.1-mini	**86.90**	82.86	**91.99**	91.93
GPT-4.1-mini	GPT-5-mini	**84.03**	79.09	**90.13**	90.13
GPT-4.1-mini	Claude Haiku 5.5	**80.93**	77.06	**87.42**	88.03
GPT-4o-mini	GPT-4o-mini	**79.35**	75.39	**86.47**	82.25

G LoCoMo-Refined: protocol and further results

Data. LoCoMo-Refined [41] (commit `887091190789`, CC BY-NC 4.0) holds LoCoMo's ten conversations (272 sessions, 5,882 turns) and 1,382 questions in four categories (213 multi-hop, 299 temporal, 68 open-domain and 802 single-hop), 521 of them marked multimodal. Against LoCoMo's 1,540, 1,278 keep their question and answer, 73 keep the question with a new gold answer, 31 are changed or new, and 178 are dropped. In all, 163 questions list more than one acceptable answer.

Stores and systems. Every store is rebuilt from the benchmark's own conversation file. MUSTER applies the rule of Section 3 unchanged and reads ten units. Mem0 2.1.0 runs as a self-hosted server and extracts with GPT-OSS-120B (reasoning effort low, served by DeepInfra through OpenRouter), every extraction prompt carrying its session date (5,882 calls, 5,359 memories), and its list is cut to 4,900 tokens. Hindsight 0.10.3 retains with the same model and service (543 calls) and is read at its default recall. True Memory 0.7.6.2 runs its Pro configuration with its HyDE expansion generated by the same model (1,382 calls) and without it, and its 100 results are cut, in rank order, to 4,900 tokens. EverMemOS runs its released code at `fe436d3` with GPT-OSS-120B (reasoning effort low, served by vLLM on one RTX PRO 6000) for construction and agentic retrieval, beside Qwen3-Embedding-4B and Qwen3-Reranker-4B, and is read at its ten retrieved memory cells (582 cells, with 6,940 construction and 2,094 retrieval calls counted together as on LoCoMo). HippoRAG-v2 runs as on LoCoMo, its graph and per-question fact filter on GPT-OSS-120B (1,926 calls), and is read at its ranked sessions filled to 4,900 tokens. No system ingests an image's search query or URL. The query holds the gold answer in 75 questions. Captions are kept where a system's loader takes them. Honcho runs as its hosted service under the LoCoMo runner of Appendix F, with no image URL or search query in any message's metadata, in new workspaces built from the benchmark's file (5,882 messages, a shared image's caption appended to its message, and one dream per speaker once the queue has drained). Honcho's context API is given 4,900 tokens and returns 5.2K, and Honcho

Table 26: **LoCoMo-Refined by category.** Categories under the official judge [41], and overall under the official and AML [42] judges. Bold: MUSTER.

	Multi-hop	Temporal	Open-domain	Single-hop	All	
Store read, retrieval	(213)	(299)	(68)	(802)	official	AML judge
***reader**: GPT-4o-mini*						
Mem0 2.1.0	56.81	66.22	64.71	81.80	73.73	71.85
Honcho	47.42	35.12	57.35	66.71	56.44	54.56
Hindsight 0.10.3	52.58	57.86	63.24	68.83	63.68	61.94
EverMemOS	59.62	63.55	63.24	84.91	75.33	73.23
HippoRAG-v2	50.23	39.46	69.12	82.54	67.58	65.56
True Memory, HyDE	57.75	52.51	63.24	83.04	71.56	70.19
True Memory, no HyDE	61.50	51.51	63.24	82.67	71.71	70.19
Full store	61.97	35.45	67.65	88.78	72.07	69.90
MUSTER, $U{=}10$	**62.91**	**54.18**	**75.00**	**90.40**	**77.57**	**75.69**
***reader**: GLM-5.3-Flash*						
Mem0 2.1.0	68.08	74.58	77.94	86.28	80.54	77.93
Honcho	58.69	53.51	60.29	72.32	65.56	62.66
Hindsight 0.10.3	61.50	68.23	73.53	74.19	70.91	68.81
EverMemOS	66.67	71.91	80.88	87.03	80.32	78.51
HippoRAG-v2	60.56	63.55	80.88	84.41	76.05	74.46
True Memory, HyDE	74.65	60.20	82.35	88.78	80.10	78.58
True Memory, no HyDE	66.20	60.87	79.41	88.15	78.44	77.28
Full store	74.65	85.95	91.18	93.64	88.93	87.05
MUSTER, $U{=}10$	**75.12**	**83.95**	**82.35**	**94.76**	**88.78**	**87.41**
***reader**: Qwen3.5-9B*						
Mem0 2.1.0	48.36	59.20	54.41	78.30	68.38	66.79
Honcho	40.38	28.09	48.53	64.34	52.03	49.71
Hindsight 0.10.3	46.01	38.13	48.53	66.33	56.22	55.35
EverMemOS	50.70	55.52	51.47	80.92	69.32	67.22
True Memory, HyDE	46.95	31.10	47.06	78.80	62.01	60.71
True Memory, no HyDE	47.42	33.44	51.47	78.80	62.81	61.00
MUSTER, $U{=}10$	**48.83**	**34.78**	**58.82**	**87.16**	**68.52**	**66.43**

writes with the service's own models, which are not version-pinned. The whole conversation is each conversation's stored text, unretrieved (31.1K tokens per question). One write model for every memory system but Honcho (Table 31) follows the check on BEAM-100K (Appendix C). There, writing with GPT-OSS-120B in place of the model of each system's paper moves Mem0 by -0.2 and EverMemOS by 0.0 with GLM-5.3-Flash, and by $+1.0$ and -1.3 with Qwen3.5-9B under our answer prompt.

Answering and judging. Every context fills the first memory slot of the answer template of the leaderboard's public LoCoMo-Refined pipeline [42]. Readers answer at temperature 0. GPT-4o-mini (`gpt-4o-mini-2024-07-18`), the base model of every academic result on the leaderboard, answers through OpenAI's API, and the two readers of our BEAM runs (Section 5.4), GLM-5.3-Flash at maximum reasoning effort and Qwen3.5-9B with reasoning off (0 reasoning tokens in every answer), answer through OpenRouter with one pinned provider each. Claude Sonnet 5.5 (`claude-sonnet-5-5`) answers through Anthropic's API at the API's defaults, as the model accepts no temperature, with adaptive thinking and at most $8{,}192$ output tokens. The official judge is Qwen3-14B (bf16, revision `40c06982`, temperature 0, thinking off) served with vLLM 0.30.0. Qwen3-14B judges each acceptable answer in its own call and counts an exact match as correct without one. The AML judge, the official prompt with one added line and all acceptable answers in one call, is reported beside it. Paired comparisons use the exact two-sided McNemar test.

Results. Table 26 gives each category and both judges for the stores and contexts above under GPT-4o-mini, GLM-5.3-Flash and Qwen3.5-9B, with no full-store or HippoRAG-v2 row for Qwen3.5-9B, and Table 27 the paired counts of all four readers. A second GPT-4o-mini run of MUSTER scores 77.13 (240 answers differ, net 6 questions). With other readers MUSTER scores 78.80 with GPT-4.1-mini and 83.21 with GPT-5-mini at its default reasoning. True Memory's HyDE moves its score by -0.14, $+1.66$ and -0.80 points with GPT-4o-mini, GLM-5.3-Flash and Qwen3.5-9B ($p = 0.93$, 0.065 and 0.37).

Reports by others. The benchmark's README lists MemoraX at 82.65, a new run that its maintainer reports with GPT-4o-mini for memory construction and answering and with an answer prompt

Table 27: **Paired counts on LoCoMo-Refined.** MUSTER (U=10) against each row at the same reader, with questions only one side answers correctly under the official judge [41] and the exact two-sided McNemar p. Bold: MUSTER.

Other row	**Only MUSTER**	Only the other	p
***reader**: GPT-4o-mini*			
Mem0 2.1.0	**182**	129	0.003
Honcho	**360**	68	< 0.001
Hindsight 0.10.3	**294**	102	< 0.001
EverMemOS	**144**	113	0.061
HippoRAG-v2	**205**	67	< 0.001
True Memory, HyDE	**168**	85	< 0.001
True Memory, no HyDE	**177**	96	< 0.001
Full store	**143**	67	< 0.001
***reader**: GLM-5.3-Flash*			
Mem0 2.1.0	**177**	63	< 0.001
Honcho	**370**	49	< 0.001
Hindsight 0.10.3	**314**	67	< 0.001
EverMemOS	**175**	58	< 0.001
HippoRAG-v2	**238**	62	< 0.001
True Memory, HyDE	**182**	62	< 0.001
True Memory, no HyDE	**194**	51	< 0.001
Full store	**62**	64	0.93
***reader**: Claude Sonnet 5.5*			
Mem0 2.1.0	**156**	72	< 0.001
Honcho	**345**	48	< 0.001
Hindsight 0.10.3	**294**	66	< 0.001
EverMemOS	**145**	66	< 0.001
HippoRAG-v2	**228**	52	< 0.001
True Memory, HyDE	**137**	63	< 0.001
True Memory, no HyDE	**133**	59	< 0.001
Full store	**62**	73	0.39
***reader**: Qwen3.5-9B*			
Mem0 2.1.0	**178**	176	0.96
Honcho	**330**	102	< 0.001
Hindsight 0.10.3	**305**	135	< 0.001
EverMemOS	**157**	168	0.58
True Memory, HyDE	**183**	93	< 0.001
True Memory, no HyDE	**187**	108	< 0.001

it does not disclose, beside earlier predictions of MemOS (63.60), MemPalace (58.68), EverMemOS (58.25) and Mem0 (48.91) re-scored with the same judge [41]. The Agent Memory Leaderboard [42], which uses this set, publishes overall and capability scores but no per-dataset scores.

H WHERE THE SLOTS COME FROM

On the two FactConsolidation cells, each question's selection (about ten records, each with its recorded serial) is fixed, and only the source of the supersession slots changes. Unlike MUSTER's prune view, which substitutes from the whole store (Section 3.5), the test resolves within the fixed selection. A record is dropped when one of its facts has a newer value for the same slot in another record. The kept records are shown in ascending serial with no refill, and GPT-4o-mini reads them at temperature 0. The byte-read slot is the harness key of Appendix A. The extractors see the same records, each cut to 600 characters. One is an attribute-keyed prompt, "From the numbered memory passages below, extract facts ABOUT THE USER relevant to this question: $\langle q \rangle$. Output one per line exactly as: attribute :: value :: passage_number. Include superseded/older values too. Max 25 lines." with the lower-cased attribute as the slot, and the other an entity-aware prompt keyed by entity and attribute, "From the numbered passages below, extract every fact relevant to this question: $\langle q \rangle$. Output one per line exactly as: entity :: attribute :: value :: passage_number. The entity is the subject the fact is about. Include superseded/older values too. Max 25 lines." Without the refill of MUSTER's read path and at temperature 0, the byte-read row scores 41 on FC-MH where Table 1's refilled, temperature-0.7 run scores 37.0.

Table 28: **Byte-read slots match model extraction.** Slot source on FactConsolidation's fixed selections, 100 questions per cell. Model calls and dropped records are per question. Bold: MUSTER.

Slot source	model calls	dropped	FC-MH	FC-SH
Read from the bytes (MUSTER)	**0**	**0.89**	**41**	**92**
Attribute-keyed prompt, GPT-4o-mini	1	2.40	37	90
Entity-aware prompt, GPT-4o-mini	1	1.34	42	91
Entity-aware prompt, GPT-4.1-mini	1	0.65	43	93
byte-read slots, order changed				
Recorded serials shuffled among the records	0	0.89	33	59
No order: all records, no serials, random order	0	0	35	64

Reading the slots from the bytes reaches the accuracy of an entity-aware extractor without a model call (Table 28). The slots are read against the order. Permuting the same serial values among each question's records (one seed per question) lowers FC-SH from 92 to 59 (37 questions worse, 4 better) and FC-MH from 41 to 33 (10 against 2), below showing every record with no order at all. A wrong order can cost only units that share a slot. Were every record of the twelve stores the rule reads as non-serial numbered with no order, keep-newest would drop 35 of their 61,156 records (0.06%, and 1.7% of their lines) whose text differs from the record kept, against 30.2% of the lines of FactConsolidation, where the numbers are an order. The attribute-keyed prompt drops 2.7× as many records, because facts about different entities share an attribute string.

I THE READ PATH AGAINST A RANKING-ONLY CONTROL, AND THE THREE VIEWS

Table 29: **Read path against ranking alone.** MemoryAgentBench cells with GPT-4o-mini. Full: MUSTER's read path. Rank: the cross-encoder ranking of the same detected units. Δ: full minus rank. Bold: MUSTER.

Cell	rank	**full**	Δ	Cell	rank	**full**	Δ	Cell	rank	**full**	Δ
FC-MH	.120	**.370**	+.250	LongMemEval	.540	**.563**	+.023	MH-QA	.650	**.650**	.000
EventQA	.660	**.820**	+.160	SH-QA	.900	**.920**	+.020	Recsys	.132	**.133**	+.001
DetectiveQA	.592	**.662**	+.070	NLU	.860	**.880**	+.020	CLINC150	.910	**.900**	−.010
TREC-coarse	.660	**.700**	+.040	Banking77	.940	**.960**	+.020	TREC-fine	.690	**.650**	−.040
				FC-SH	.900	**.920**	+.020				

Table 30: **recshare chooses the unit on eight cells.** MemoryAgentBench at $U = 10$ units before compensated refinement (Section 3.5), GPT-4o-mini. Bold: the unit recshare picks. -0.010, within 0.010.

Cell	recshare	recshare picks	refined 512	protocol 4096	pick better?
Banking77	1.0000	refine	**0.960**	0.930	yes
TREC-coarse	1.0000	refine	**0.700**	0.590	yes
NLU	1.0000	refine	**0.880**	0.890	−0.010
TREC-fine	1.0000	refine	**0.650**	0.570	yes
CLINC150	1.0000	refine	**0.900**	0.910	−0.010
EventQA	0.0000	keep	0.710	**0.750**	yes
DetectiveQA	0.0000	keep	0.535	**0.624**	yes
Recsys	1.0000	refine	**0.133**	0.147	*no*

The three views. The three views are produced in one run. *Prune* keeps the newest unit per slot, refills, and renders in ascending serial. *Annotate* keeps every unit, in descending serial, and prefixes the newest per slot with a current-value tag. *Label* is annotate with the superseded units given a superseded tag, so annotate and label share unit set, order and budget and both keep the history that removal discards. Each unit carries its serial. MUSTER uses prune and shows each record as `[serial] text`.

J FURTHER RESULTS

The slot key barely changes the result. Four keys, from the words before a record's first form of be or have to the key MUSTER uses (Appendix A), all sit between 0.370 and 0.380 on FC-MH (mean of three reads each), although the first leaves a fifth of the records without a slot. MUSTER's key is derived from the bytes rather than looked up by dataset name. An extractor keyed by attribute alone loses 4 points on FC-MH in the fixed-selection setting (Appendix H).

Every MUSTER row of Table 1 holds the protocol allowance. On InfBench-Sum, forty 1024-token units read $40{,}326$ tokens per question and ten protocol chunks read $40{,}325$ (mean over the 100 questions).

J.1 ONE STATISTIC, OPPOSITE CHOICES

A statistic that always picks the same unit cannot be checked, so Table 30 in Appendix I reports both choices. The table fixes $U = 10$ units, before compensated refinement, and DetectiveQA is the mean of three reads on both sides. recshare licenses a finer cut on six of the eight cells and refuses it on two. Of the six it wins three, is within 0.010 on two and loses one, Recsys, most likely because dialogue turns are short enough to fire the statistic while the preference a question needs spans several turns. Paired over questions it separates on TREC-coarse, $+0.110$ on 17 flips against 6 ($p = 0.035$, exact two-sided sign test).

On the two cells where it refuses, keeping the protocol chunk is right by a wide margin at a fixed count. At $U = 10$ finer units cost 0.040 on EventQA and 0.089 on DetectiveQA, with an eightfold smaller allowance. The fixed-allowance test is the compensated branch, where EventQA gains 7.0, 8.0 and 12.0 over ten 4096-token units and reaches 82.0 at all three readers. One statistic therefore produces opposite decisions and is correct in both directions at fixed count.

J.2 WHAT THE READ PATH IS WORTH, AND WHERE

We test the read path apart from the statistics that configure it. Table 29 in Appendix I ablates it against the same candidates ranked by the shared cross-encoder alone, with no reserved floors, no parent and no order, on the thirteen cells where that instrument has a scored run, all but InfBench-Sum, with DetectiveQA the mean of three reads on both sides. The mean is $+0.044$, and two cells hold most of it. FC-MH's $+0.250$ comes from the serial branch, where the parent map is a real partition and the order is read. EventQA's $+0.160$ is what floors and the parent channels add together, since that cell has no serial. Floors pay where the parent map provides useful candidates, order pays only where a serial is available, and where the map is degenerate a slot reserved for a channel with nothing to contribute displaces a candidate the ranker preferred. A fixed policy pays this cost on every cell. MUSTER applies the parent-priority floors only where the bytes show a usable parent map.

Table 2 (Section 5.2) measures the same distinction from the policy side. A fixed policy must choose which population to serve, which is why the two constants land at 56.0 and 55.5. Parent-priority everywhere gains 6.0 SF points and gives up AR (70.3 against 73.8) and LRU (39.5 against 44.2). Neither policy dominates, and the gate removes the need to choose one globally.

J.3 HOW MANY ITEMS THE READER SHOULD SEE

The rank-ordered grouping stopped at ten containers because `retrieve_num` is ten, a constant that sets how many units to *fetch*, not how many containers a reader should be *shown*. The two differ here. On InfBench-Sum the forty selected units touch 16.5 parents on average (8 to 32), so ten containers must merge parents, while one container per parent keeps every container parent-pure.

At the same allowance, one container per parent instead of ten rank-ordered ones raises judged summarisation on all three readers, by $+0.044$ (61–35, $p = 0.010$, exact two-sided sign test), $+0.029$ (52–48, $p = 0.76$) and $+0.037$ (56–44, $p = 0.27$) for GPT-4o-mini, GPT-4.1-mini and GPT-5-mini, and by $+0.036$ pooled (95% CI $[+0.006, +0.067]$, 169–127, $p = 0.017$). A second read of the same contexts moves a reader's score by 0.003 to 0.039 (Appendix K), and all three readers gain. MUSTER therefore shows one container per parent on the refined branch (Section 3.5).

K RE-READ RESOLUTION

Resolution. The readers sample (temperature 0.7 for GPT-4o-mini and GPT-4.1-mini, 1.0 for GPT-5-mini), so a second read of byte-identical contexts is a second measurement. On InfBench-Sum two reads of MUSTER's contexts give no identical summary and judged F1 of 0.222 against 0.219 (GPT-4o-mini) and 0.445 against 0.417 (GPT-5-mini). With ten containers they give 0.179 against 0.198 and 0.408 against 0.369. DetectiveQA, read three times per reader on MUSTER's contexts, moves by 1.4 to 7.0 points across reads (Appendix B). Every DetectiveQA cell in this paper is therefore the mean of three reads. On BEAM-100K two reads of MUSTER's contexts score 75.3 and 74.6 at U=20 and 76.5 and 75.8 at U=40, paired differences of +0.7 and +0.6, with 279 and 284 of 400 questions scored identically. Table 5 reports the mean of the two.

L WHERE THE SETTINGS COME FROM

The two statistics (Equations 1 and 2), the thresholds $\tau_{\text{num}} = 0.80$ and $\tau_{\text{rec}} = 0.90$ and the factor of four of the compensated refinement branch (Section 3.5) were written down before the rule's runs. That branch reads four times as many quarter-size units, so the reader's allowance does not change. The read-path settings (the two floor triples, the prune view, $K_1 = 200$ and the 2048-token encoder input) were set on MemoryAgentBench, whose questions are also the reported ones. LoCoMo and BEAM reuse them. The container rule of Section 3.5 was compared on MemoryAgentBench (Appendix J.3) and BEAM before it was fixed, and the two-store assembly of Table 6 was chosen among the alternatives of Appendix C. Neither LoCoMo nor BEAM prescribes a chunk, so both take MemoryAgentBench's 4,096 tokens as c. LoCoMo reads ten units, the count MemoryAgentBench prescribes, and BEAM is reported at 20, 30 and 40 units with floors scaled to U. Our answer prompt (Appendix C) was written on BEAM's 500K conversations, a scale we do not report.

M WHO WRITES EACH MEMORY

Table 31 names the model that writes each system's memory on each benchmark. Where a system's paper reports a benchmark, the system writes with a model of that paper, and EverMemOS reproduces its reported 93.05 on LoCoMo (Appendix F). Every substitution (†) is on a benchmark the system's paper does not report, so there is no reported result of its own to reproduce. There GPT-OSS-120B, which Hindsight's paper also uses, writes the store, so these systems and Hindsight are compared under one writer, and on LoCoMo-Refined it writes every model-built store but Honcho's. GPT-OSS-120B is open-weight and runs on one RTX PRO 6000, so every such build can be rerun. Mem0 written with it scores higher than with GPT-4o-mini, the model of Mem0's paper, on MemoryAgentBench (28.6 against 26.7) and on BEAM-100K with Qwen3.5-9B under our answer prompt (63.9 against 62.9), and EverMemOS written with it scores 72.0 with GLM-5.3-Flash, as with GPT-4.1-mini, the model of its paper. Over the five BEAM-100K checks of Mem0 and EverMemOS, the change is between −1.6 and +1.0 points (Appendix C). With Qwen3.5-9B at 100K, MUSTER at 20 units scores 65.3 and 48.7 under our and BEAM's answer prompts (Table 4), above EverMemOS written with GPT-4.1-mini (63.2 and 45.8, Appendix C).

Table 31: **Who writes each memory.** The model that writes each system's store, by benchmark. †GPT-OSS-120B in place of the model of the system's paper. §Also with GPT-OSS-120B† in one row of Table 1. *GPT-OSS-20B in its primary configuration and GPT-OSS-120B in two others. ‡Our build, read in Table 7. Table 1 reports the benchmark's own run. Bold: MUSTER.

System	Its paper or service	MemoryAgentBench	LoCoMo	LoCoMo-Refined	BEAM
Mem0 2.1.0	GPT-4o-mini	GPT-4o-mini§	GPT-4o-mini	GPT-OSS-120B†	GPT-4o-mini
Honcho	service models	service models	service models	service models	service models
Hindsight 0.10.3	GPT-OSS-20B or 120B*	GPT-OSS-120B	GPT-OSS-120B	GPT-OSS-120B	GPT-OSS-120B
EverMemOS	GPT-4.1-mini	GPT-OSS-120B†	GPT-4.1-mini	GPT-OSS-120B†	GPT-OSS-120B†
HippoRAG-v2	Llama-3.3-70B-Instruct	GPT-OSS-120B†‡	GPT-OSS-120B†	GPT-OSS-120B†	GPT-OSS-120B†
True Memory	none	none	none	none	none
MUSTER	**none**	**none**	**none**	**none**	**none**